

Augmented LLMs and AI Agents

by

Rajdip Nayek

Assistant Professor,
Applied Mechanics Department,
IIT Delhi

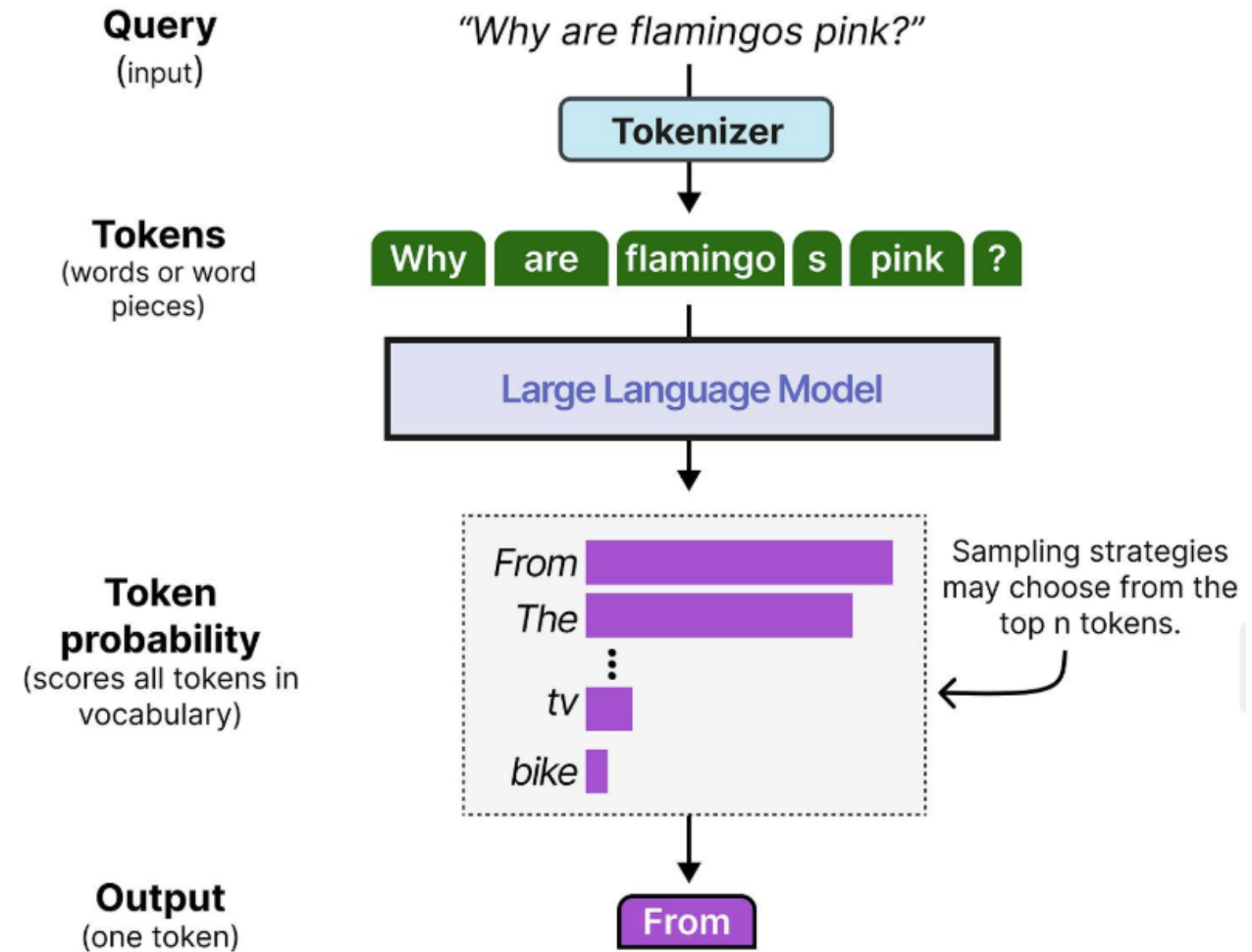
Instructor email: rajdipn@am.iitd.ac.in

Contents

- 1. What is an LLM Agent?**
- 2. Memory — short-term and long-term**
- 3. Tools — how agents act on the world**
- 4. Planning — how agents think and sequence steps**
- 5. Multi-agent collaboration**

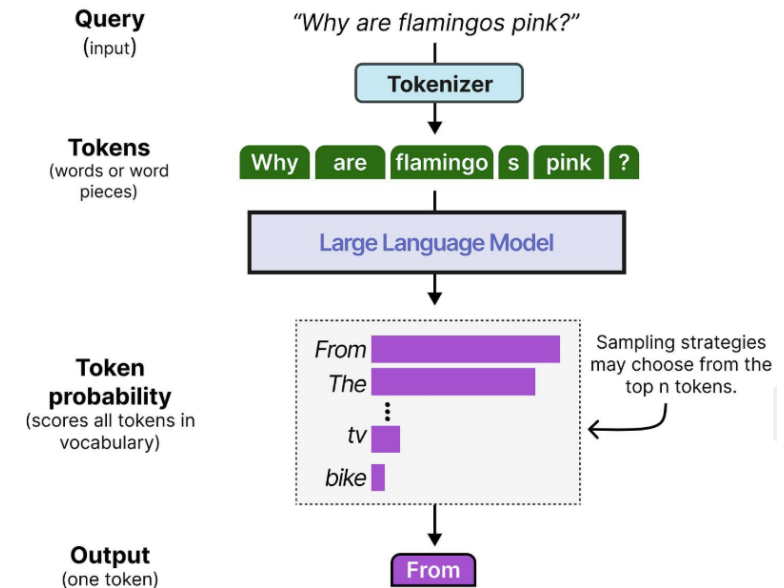
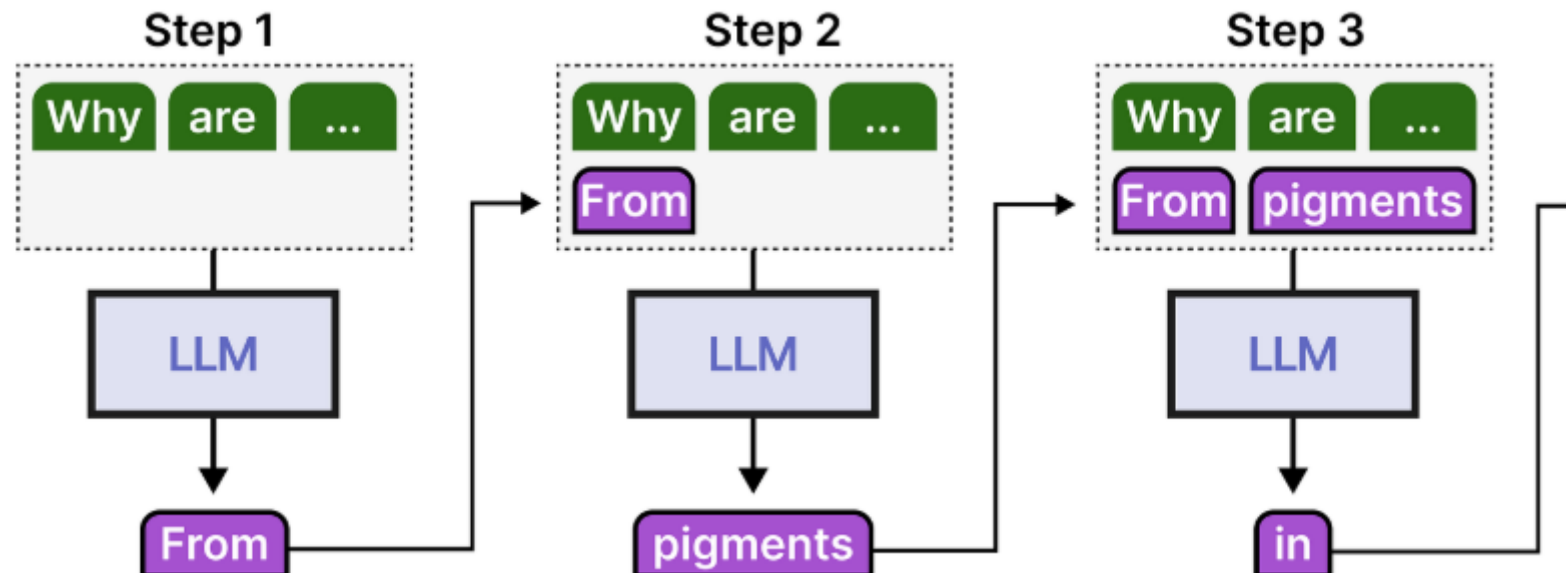
What are the basic capabilities of an LLM?

- Receives text as input
- Predicts the most likely next token



What are the basic capabilities of an LLM?

- Receives text as input
- Predicts the most likely next token
- Repeats until the response is complete



What are the basic capabilities of an LLM?

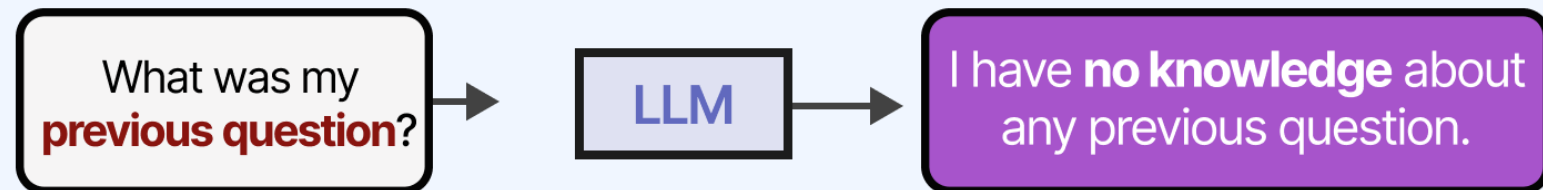
- Receives text as input
- Predicts the most likely next token
- Repeats until the response is complete
- **Works only with information available in its context window**

Conversation

Chat session 1

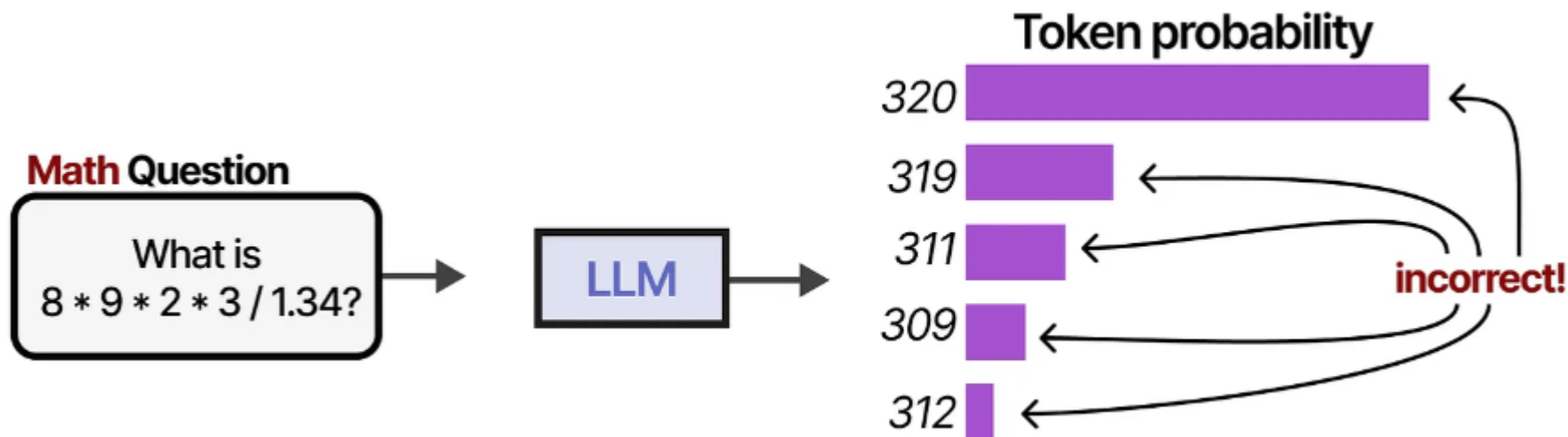


Chat session 2



Limitations of a basic LLM

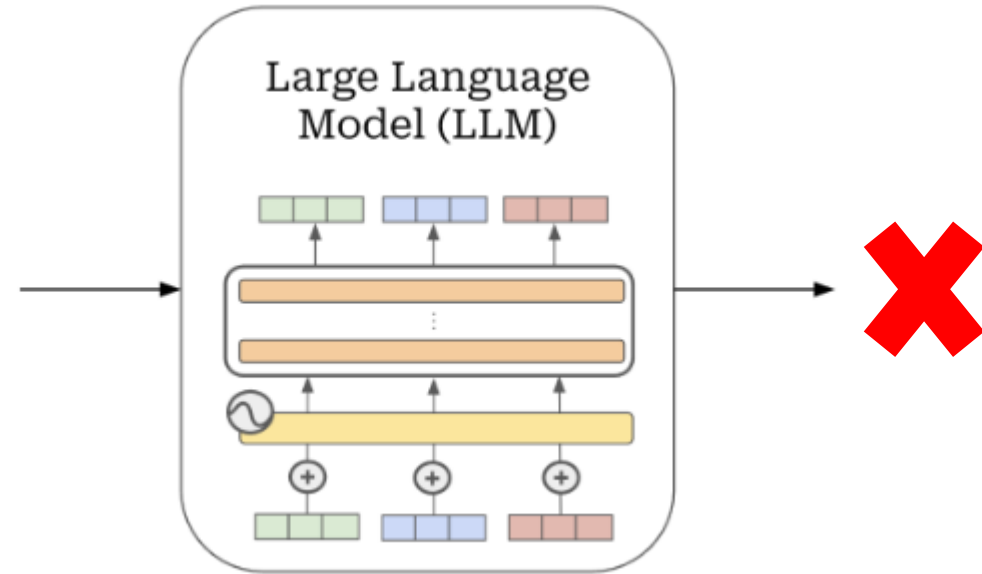
- Can get basic arithmetic wrong



Limitations of a basic LLM

- No connection to the world

Input (Prompt)
Tell me today's weather forecast



Reason it fails: The GPT model was trained last month

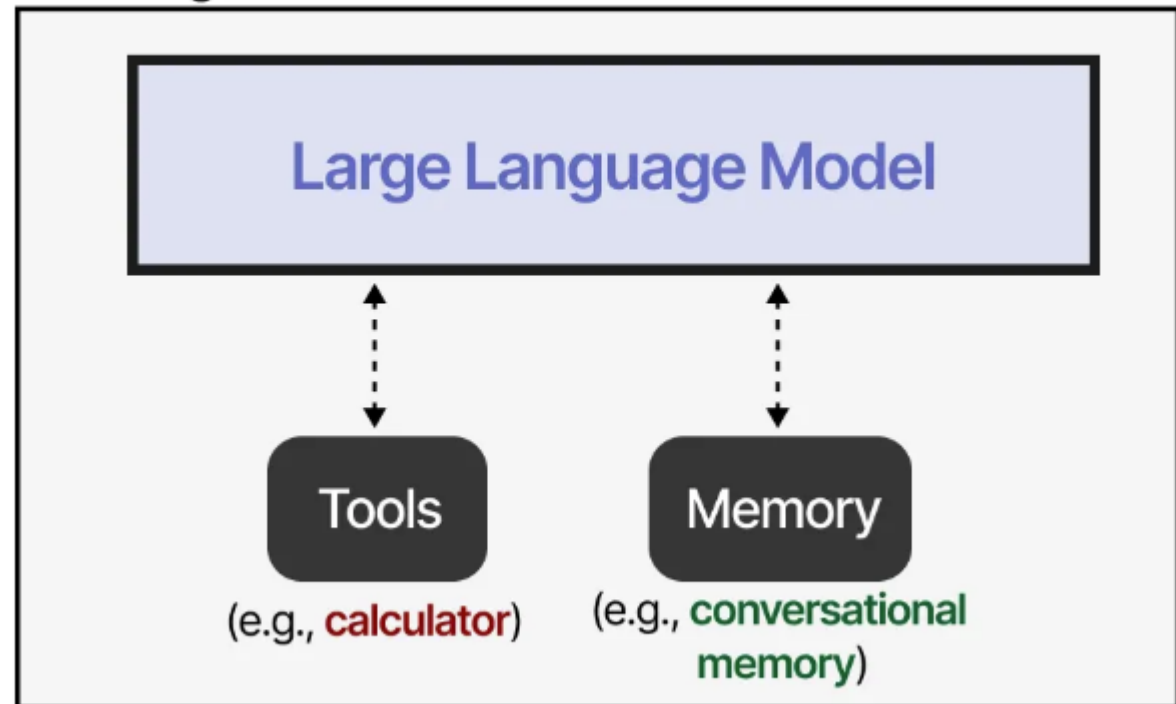
- Basic LLMs only match patterns. They do not compute. For real project numbers, this is dangerous!
- Does this mean LLMs are horrible? **Definitely not!**
- There is no need for LLMs to be capable of everything as we can compensate for their disadvantage through **external tools, memory, and retrieval systems.**

The Augmented LLM

Connect the LLM to external memory, tools, and retrieval. **Anthropic** calls this “The Augmented LLM”.

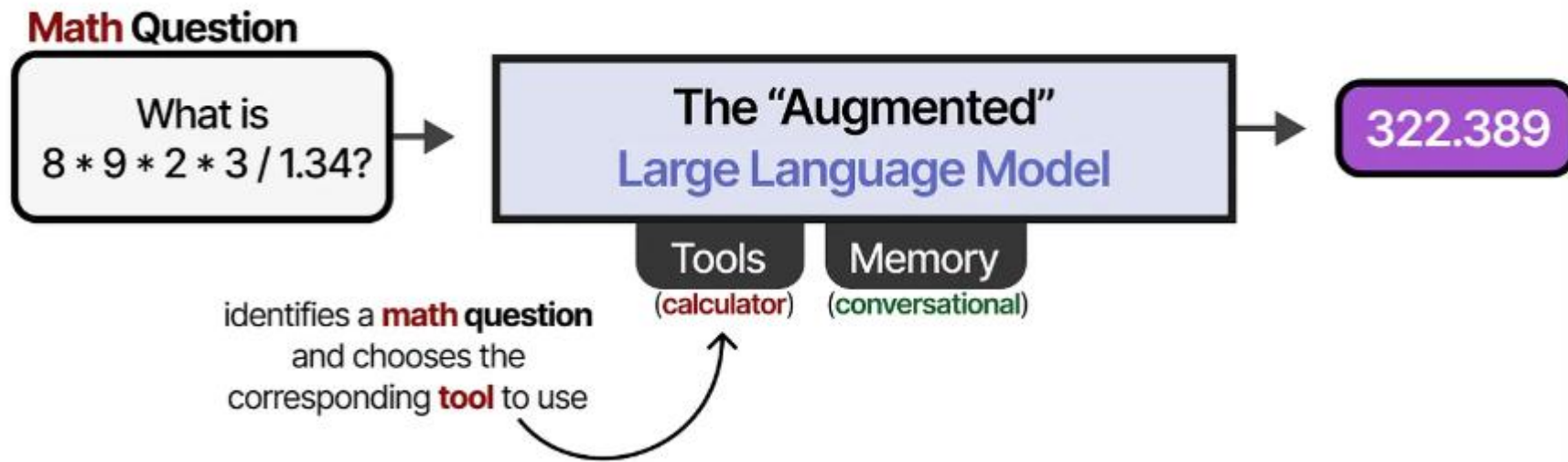
- Add **tools** for calculation and action
- Add **memory** for context
- Add **information retrieval** for up-to-date facts

The Augmented LLM



The Augmented LLM

- Ex. When faced with a math question, the LLM may decide to use the appropriate tool (a **calculator**).



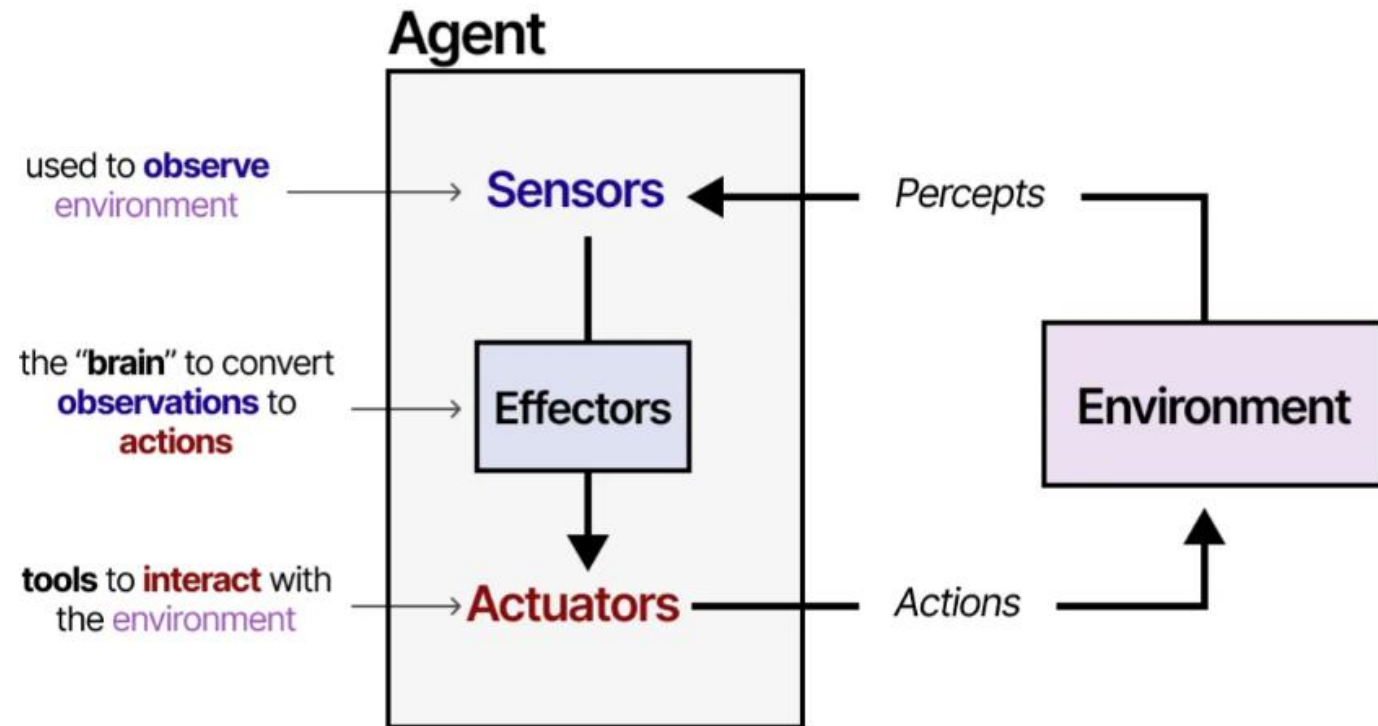
- Is this "Augmented LLM" then an Agent? No, and maybe a bit yes...

What is an Agent? The Definition

An Agent Perceives and Acts

An **agent** is anything that can be viewed as perceiving its environment through **sensors** and acting upon that environment through **actuators**.

- **Environment:** the world the agent operates in
- **Sensors:** how it observes
- **Actuators:** how it acts
- **Effectors:** the brain — decides from observation to action



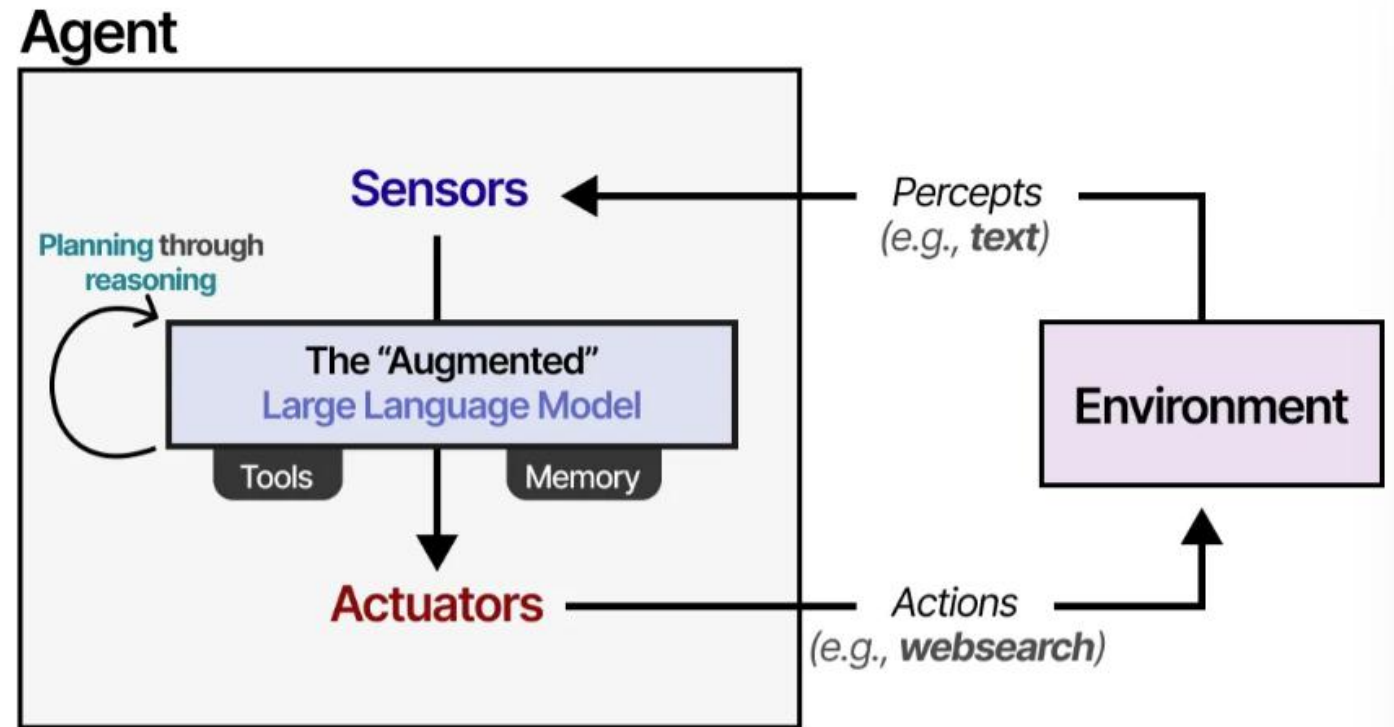
What is an **Agent**? The Definition and LLM Mapping

An Agent Perceives and Acts

An **agent** is anything that can be viewed as perceiving its environment through **sensors** and acting upon that environment through **actuators**.

For LLM Agents:

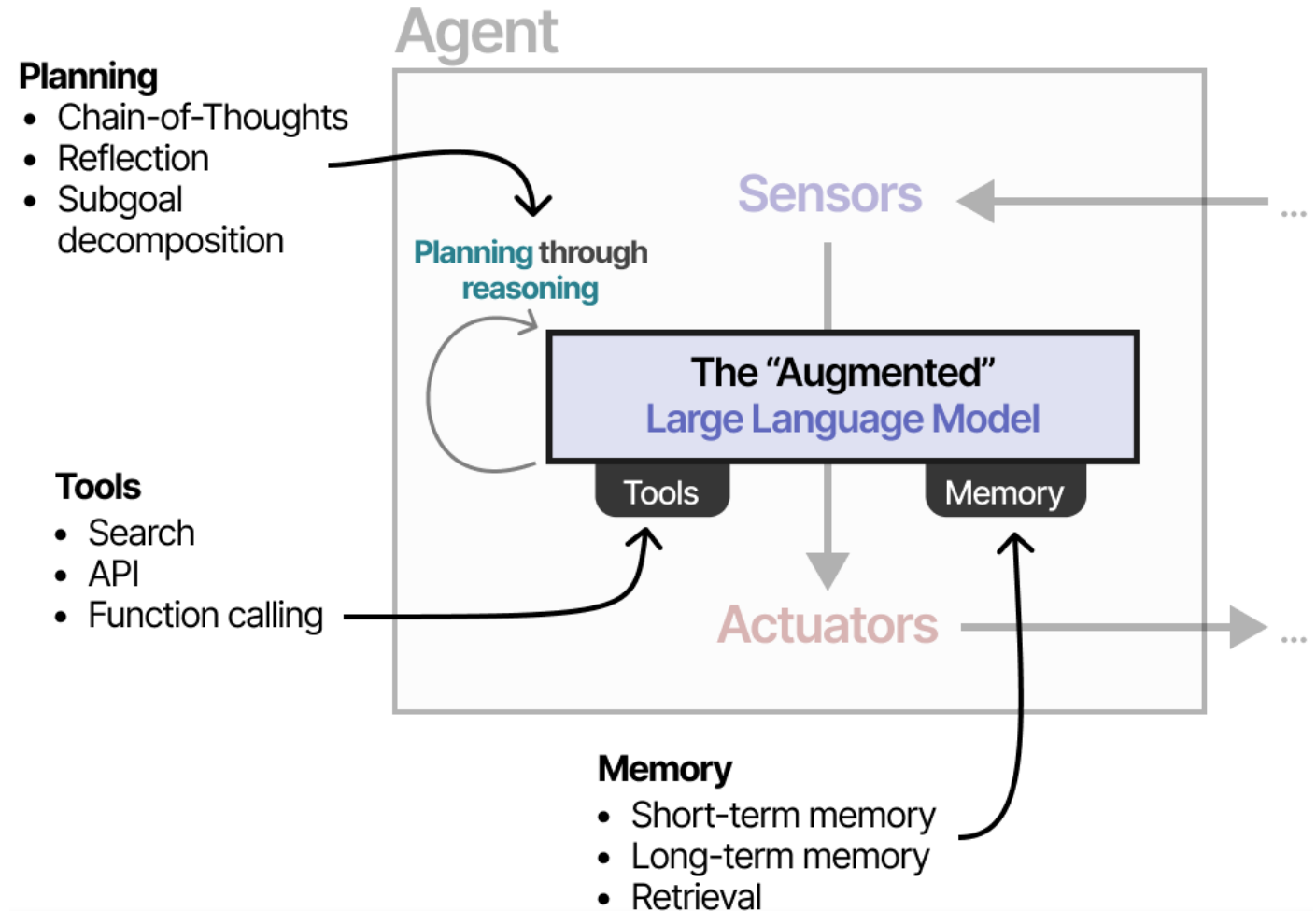
- **Environment:** digital world
- **Sensors:** textual input
- **Actuators:** tools such as web search
- **Effectors:** the brain — the LLM plus a planning component



Every Agent needs **Four Core Components**

Every LLM agent has four components working together

1. **LLM brain:** understands language and decides the next step.
2. **Planning:** breaks a big goal into a sequence of smaller actions.
3. **Tools:** reach outside text generation to get data or take action.
4. **Memory:** retain what has happened across many steps.



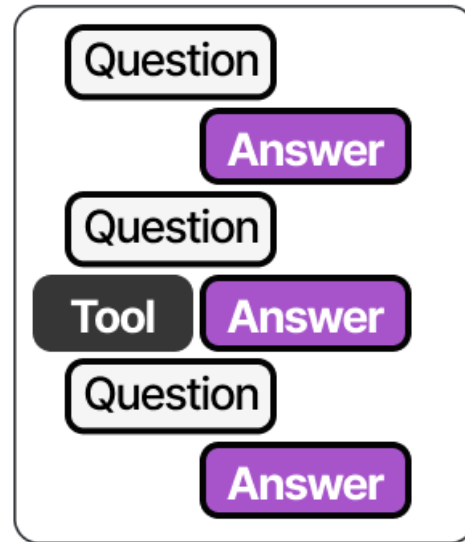
Memory

Agents need both **working memory** and **long-term memory**

- **Short-term:** current session only
- **Long-term:** persists across sessions
- Different mechanisms for storing each type of memory

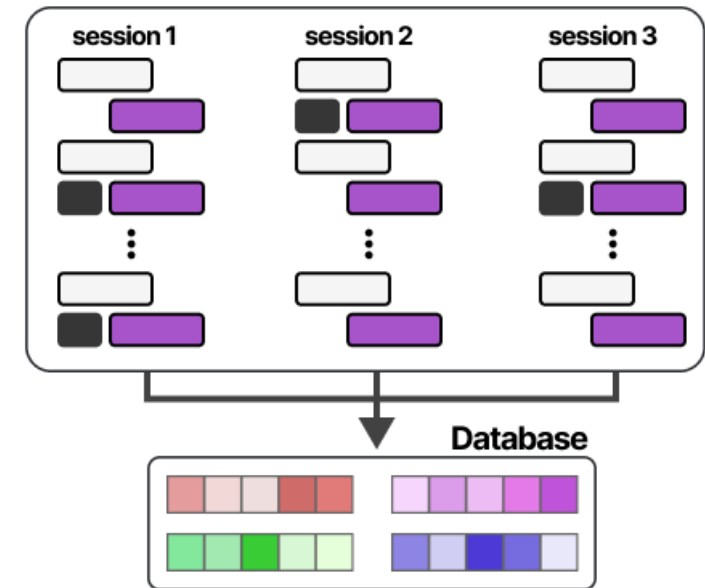
Short-term memory

(recent conversations and actions)



Long-term memory

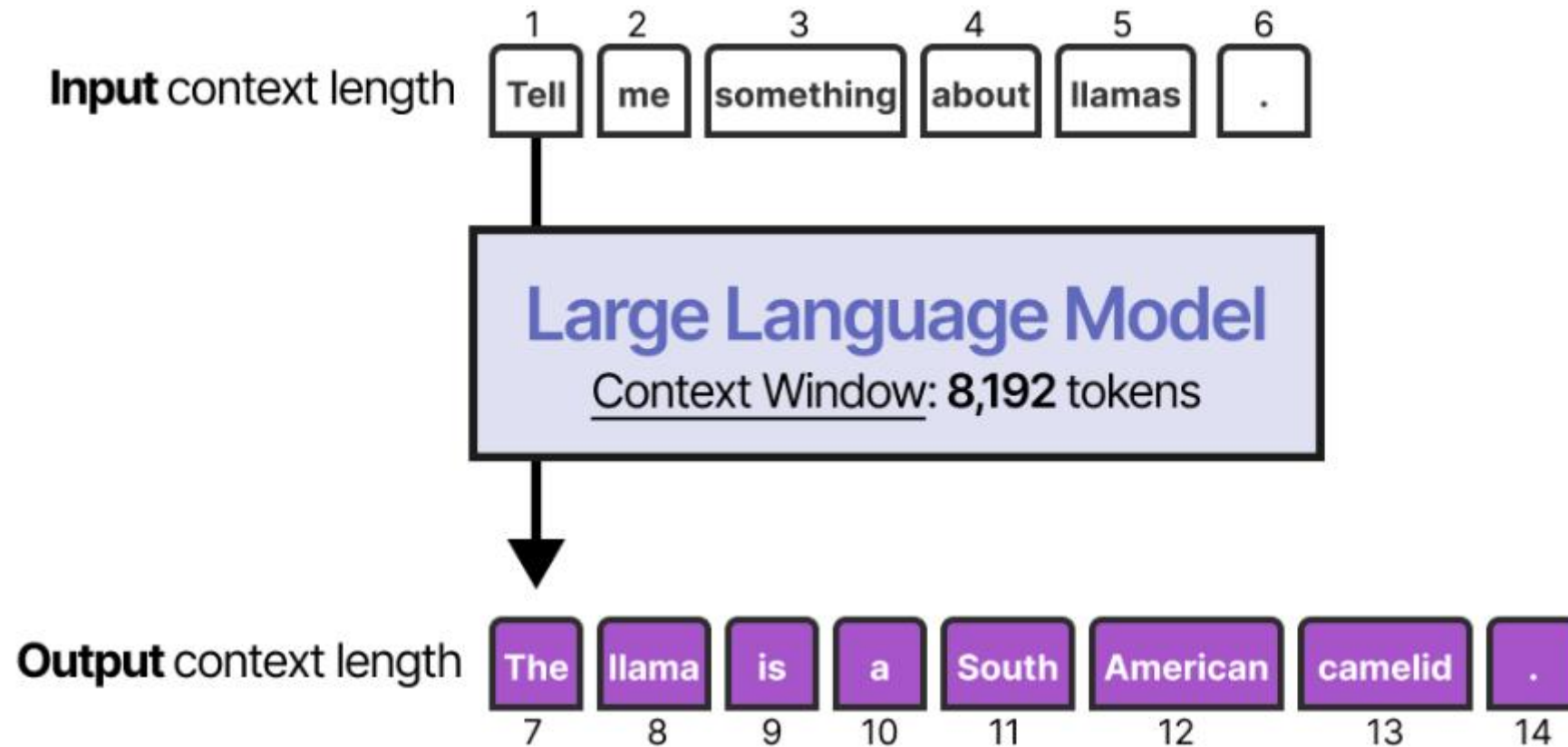
(conversations and actions over an extended period or across sessions)



LLM Agent

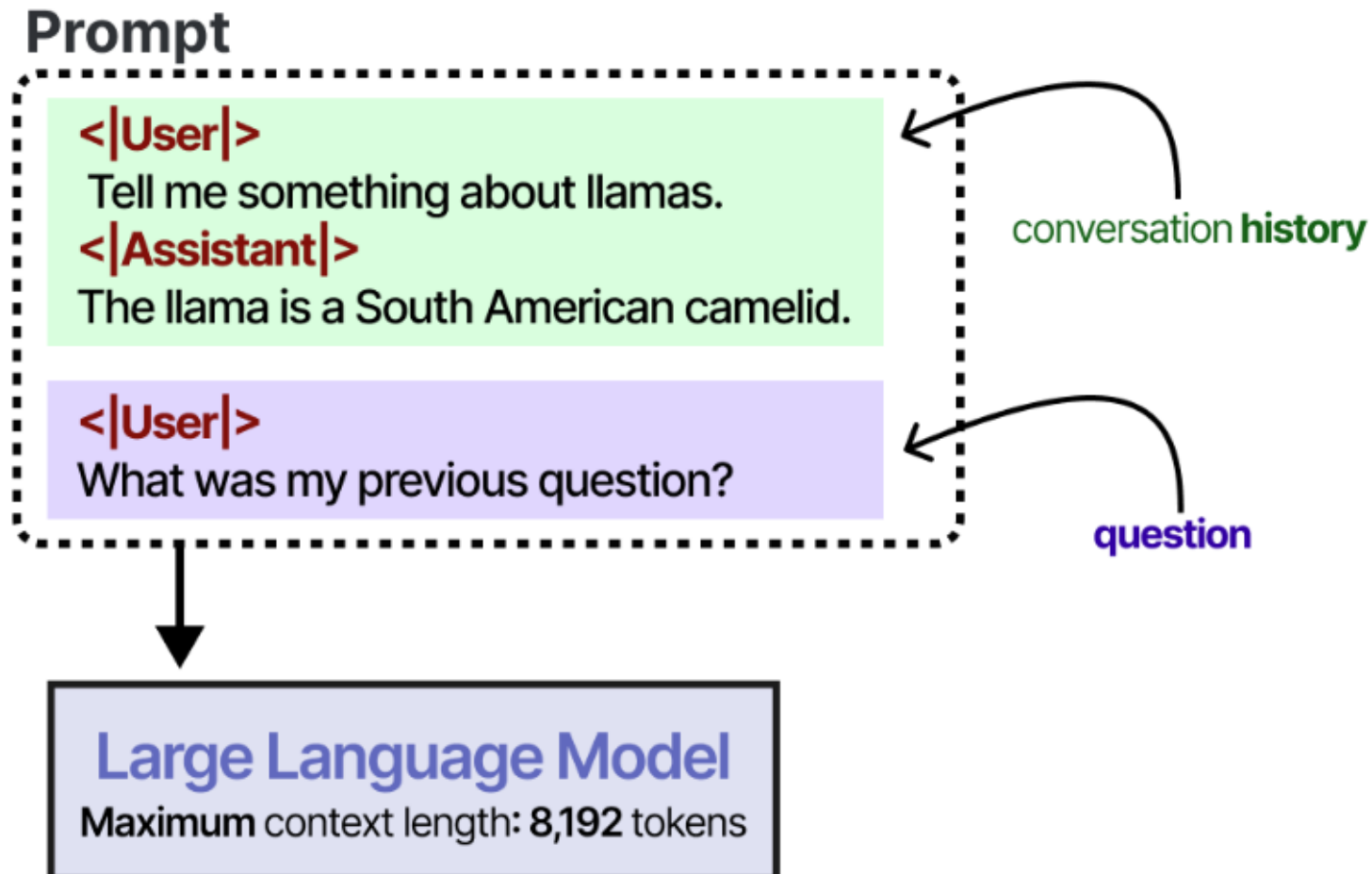
Short-term Memory: The Context Window

- Modern LLMs: **8K to 200K tokens**
- One token is roughly one word
- Full documents can be provided



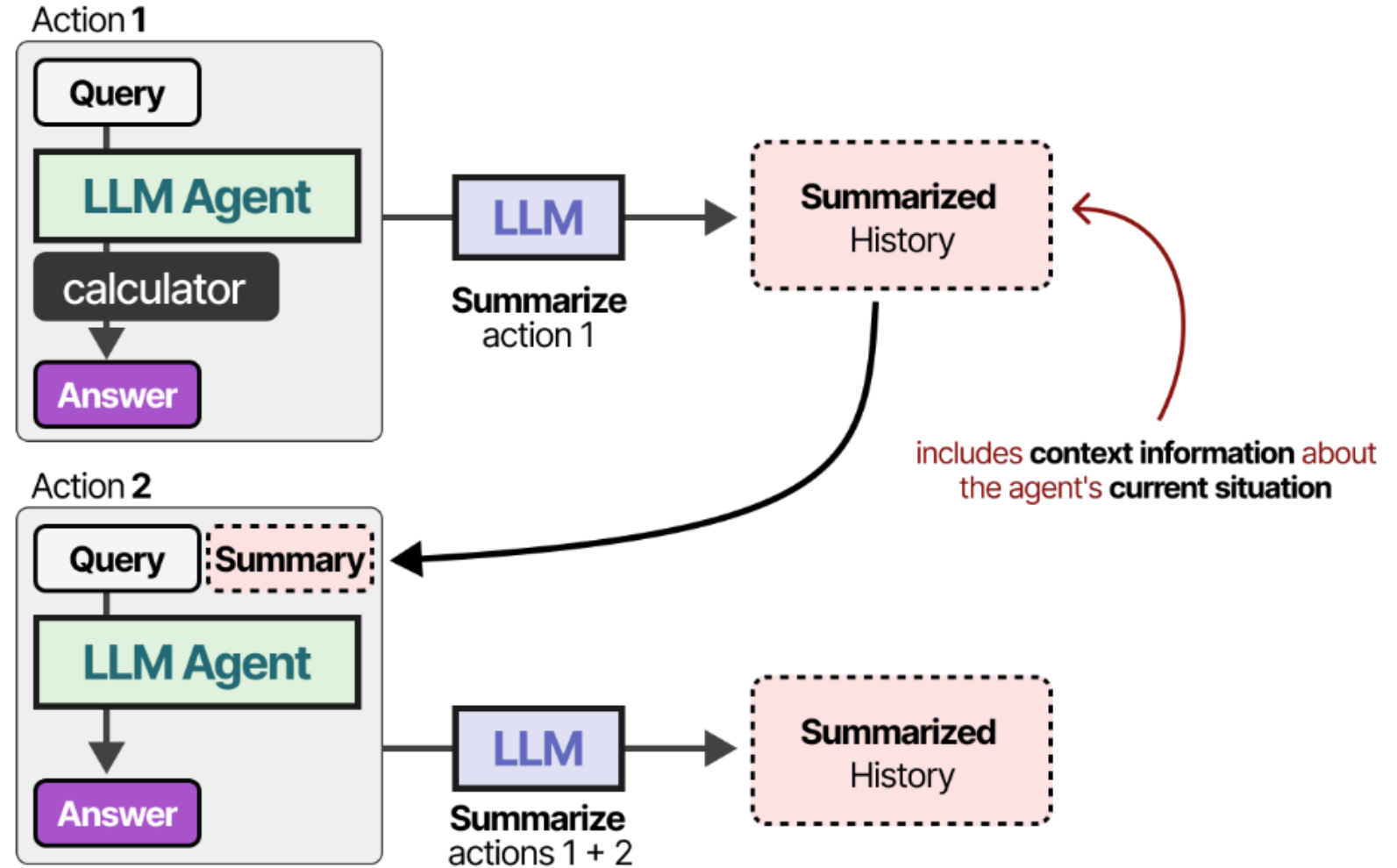
Short-term Memory: Giving the Conversation History

- All previous messages are prepended to the current message in each new LLM call for memory
- This simulates memory by telling the LLM what was said before.



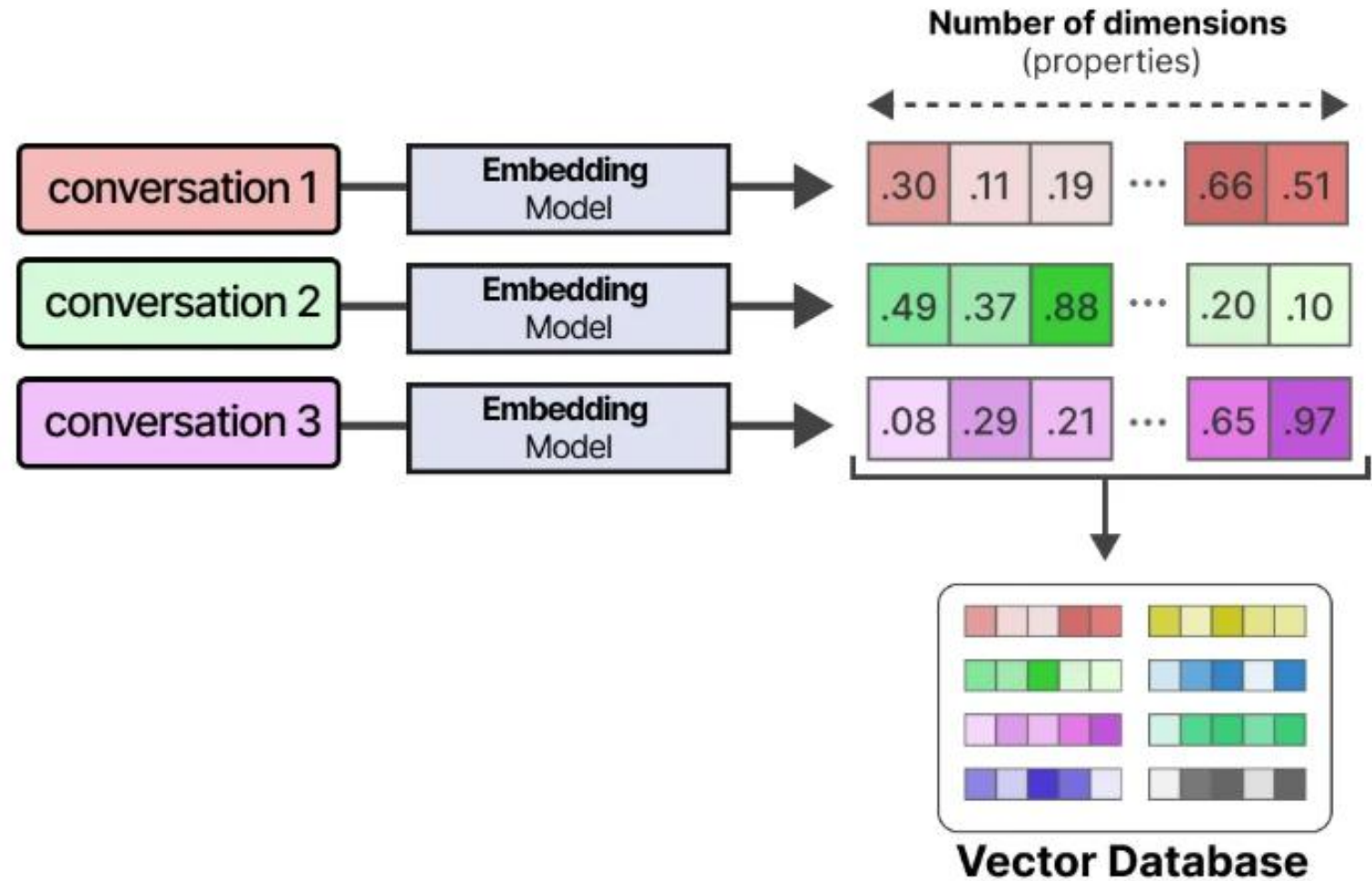
When History grow too long: Summarise

- Use an LLM to compress long conversation history into a short summary
- Long conversation history are summarised by a secondary LLM call, with the compressed summary replacing the full history on the right



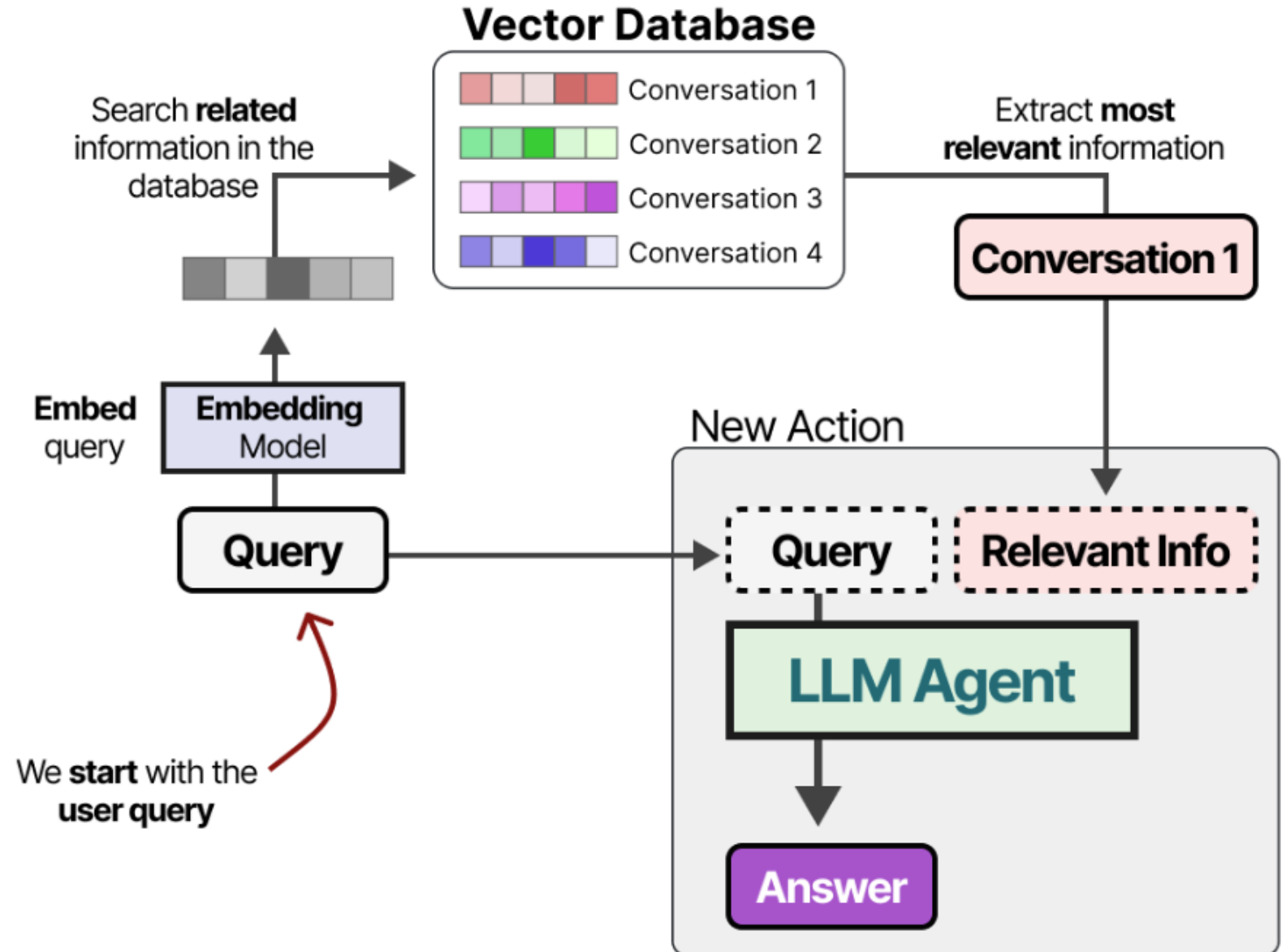
Long-term Memory: the Vector Database

- Long-term memory stores past interactions, actions, and conversations in an external vector database
- To build such a database, conversations are first embedded into numerical representations that capture their meaning.



Long-term Memory: RAG using the Vector Database

- After building the database, we can embed any given prompt and find the most relevant information in the vector database by comparing the prompt embedding with the database embeddings.
- This method is often referred to as **Retrieval-Augmented Generation (RAG)**.
- Long-term memory can also involve retaining information from different sessions.



Four Types of Memory in Agent Design

Cognitive science gives us four memory types — each maps to a different storage system

<i>Memory Type</i>		<i>Human example</i>	<i>Agent example</i>
Working	Agent's current and recent circumstances	Shopping List	Context
Procedural	Instructions to determine the agent's behavior	Tying Shoes	System Prompt
Semantic	Facts about the world	Dog Breeds	User Information
Episodic	Sequences of the agent's past behaviors	7th Birthday	Past Actions

 **Short-term** memory

 **Long-term** memory

Tools

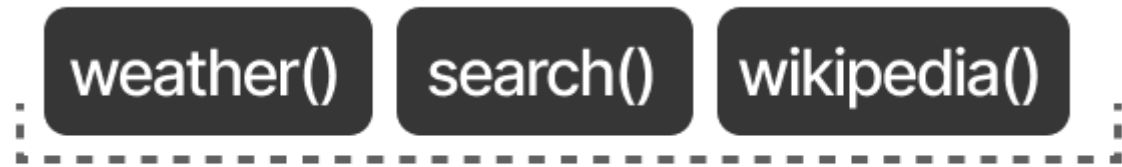
What are Tools?

Tools allow a given LLM to either interact with an external environment (such as databases) or use external applications (such as custom code to run).

Fetch Tools (read-only) – retrieve information

Examples:

- web search
- IS code lookup
- drawing register query
- live weather data

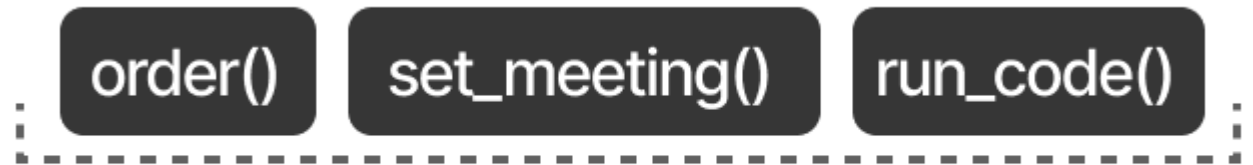


Getting **data**

Action Tools (write-capable) – change the env

Examples:

- update project register
- trigger procurement alert
- send email
- write to database
- ordering food



Taking **action**

How an LLM calls a Tool?

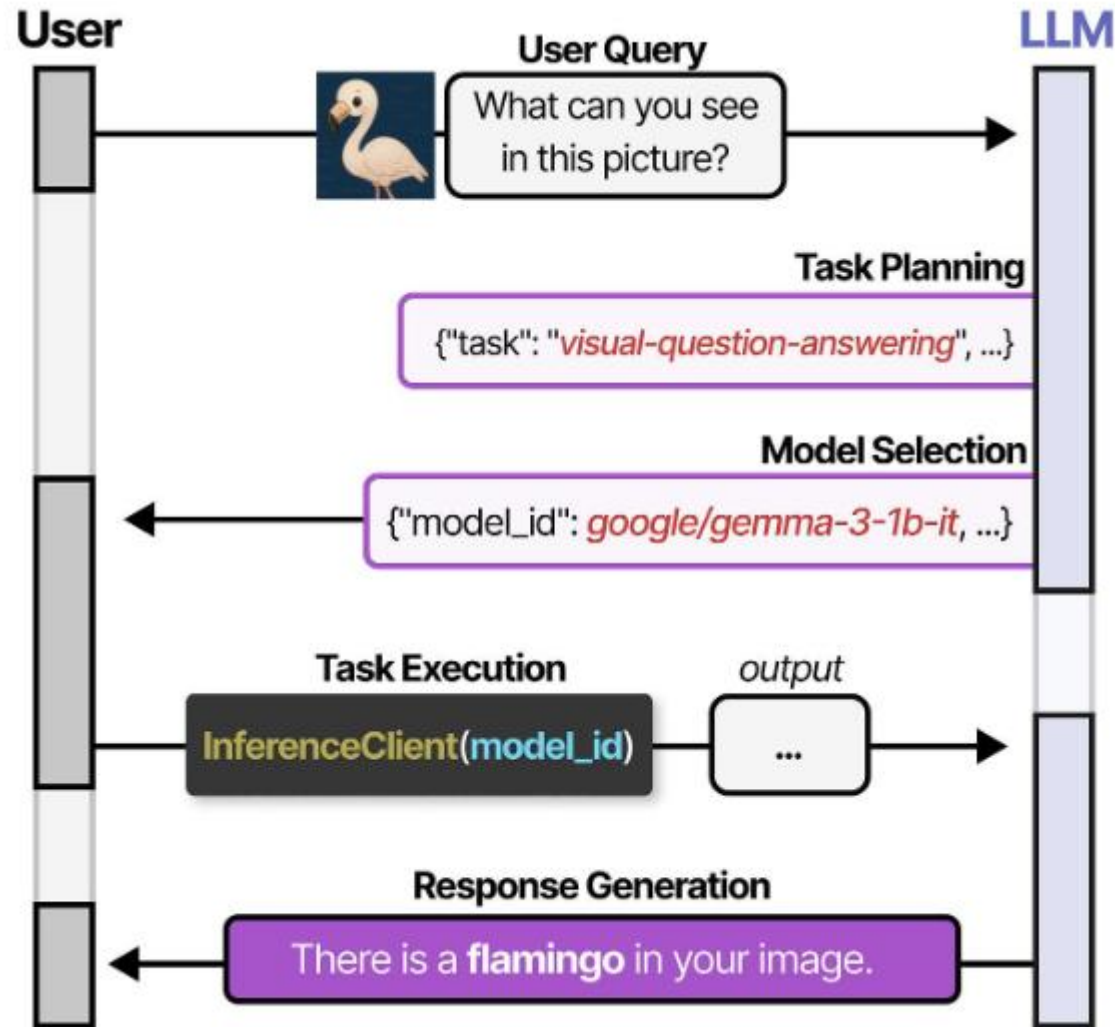
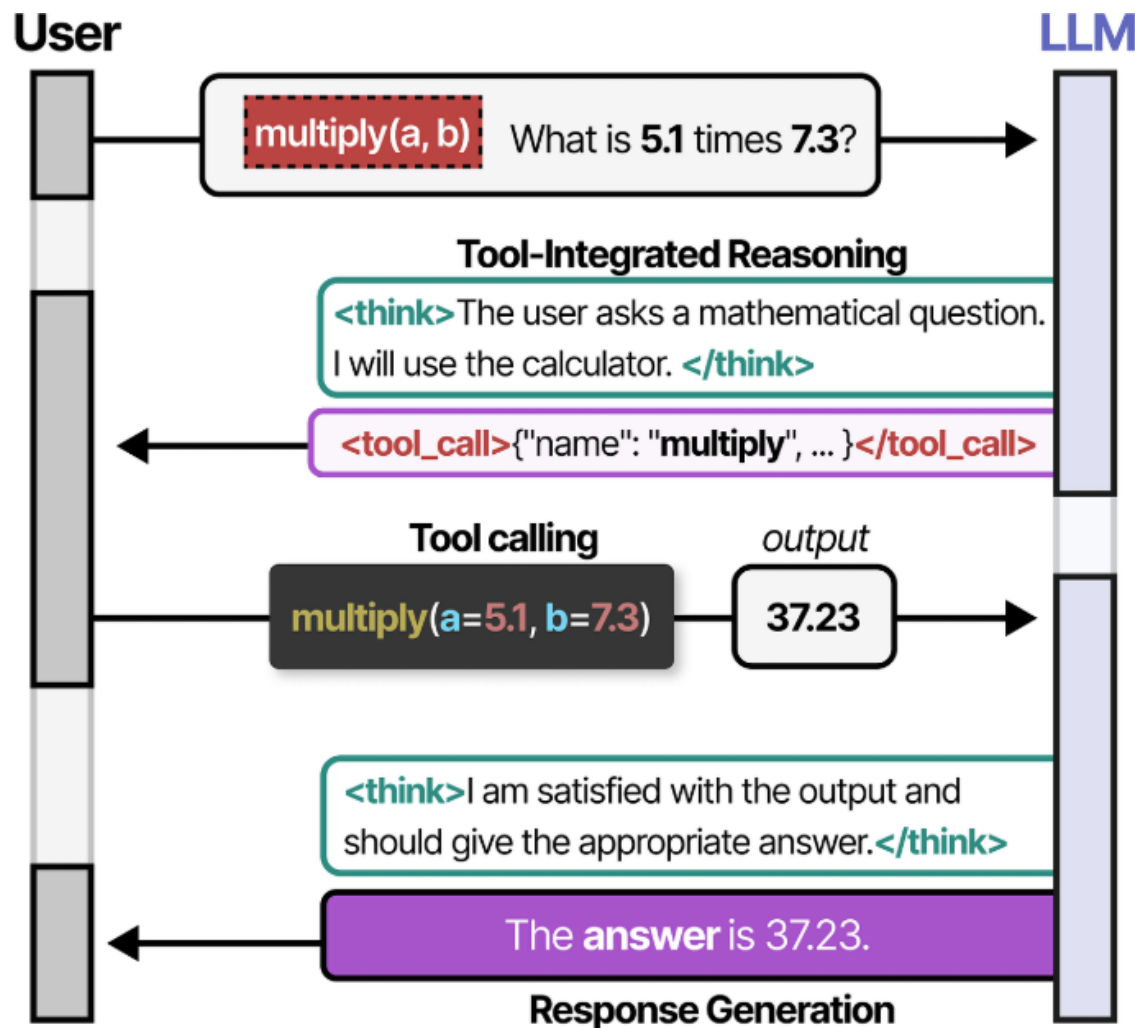
The LLM generates a structured request to the tool; the tool software code executes it

To actually use a tool, the LLM has to generate specifically formatted piece of text – typically a **JSON object** – that specified the tool name and the arguments to pass to it, so that it can easily be fed to a **code interpreter**.



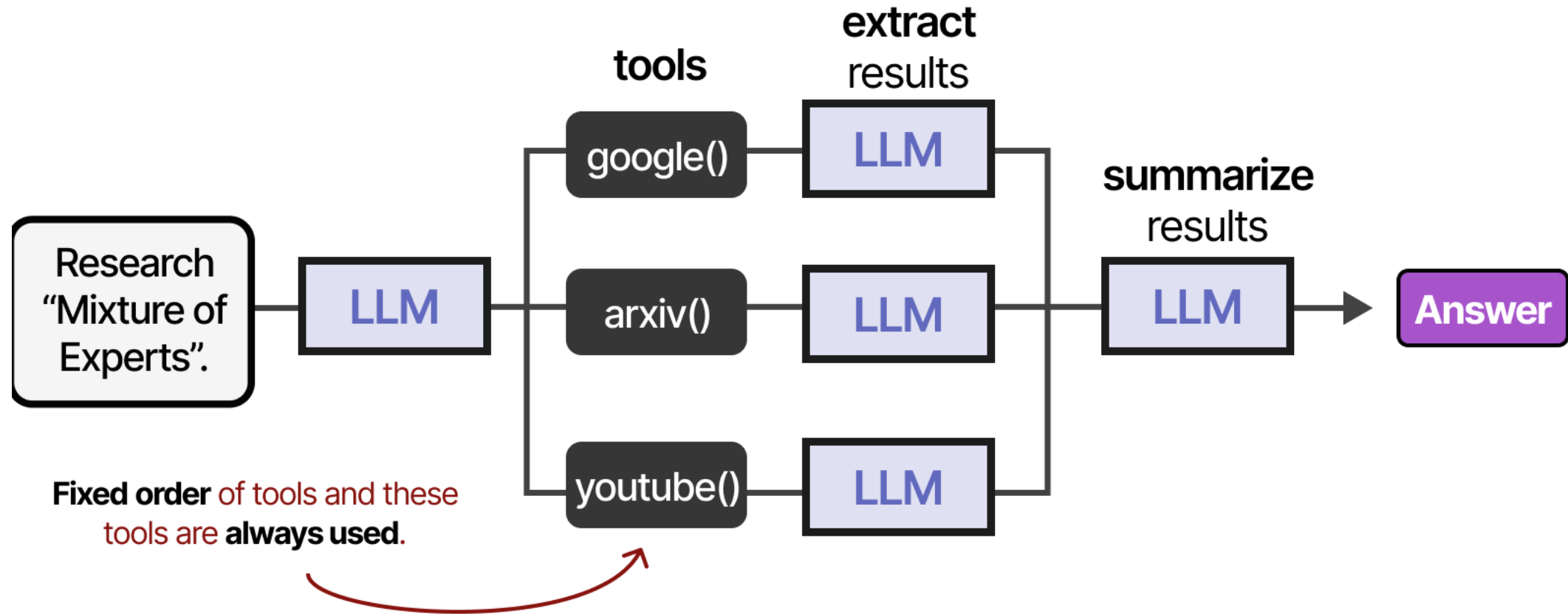
The Tool Feedback Loop

LLM generates response with tool call, code runs tool, result appended to conversation, LLM called again, continues until no tool call



Fixed pipeline vs Autonomous Choice for Tool Calling

The LLM can follow a script or choose its own path for tool selection — most deployments sit in between



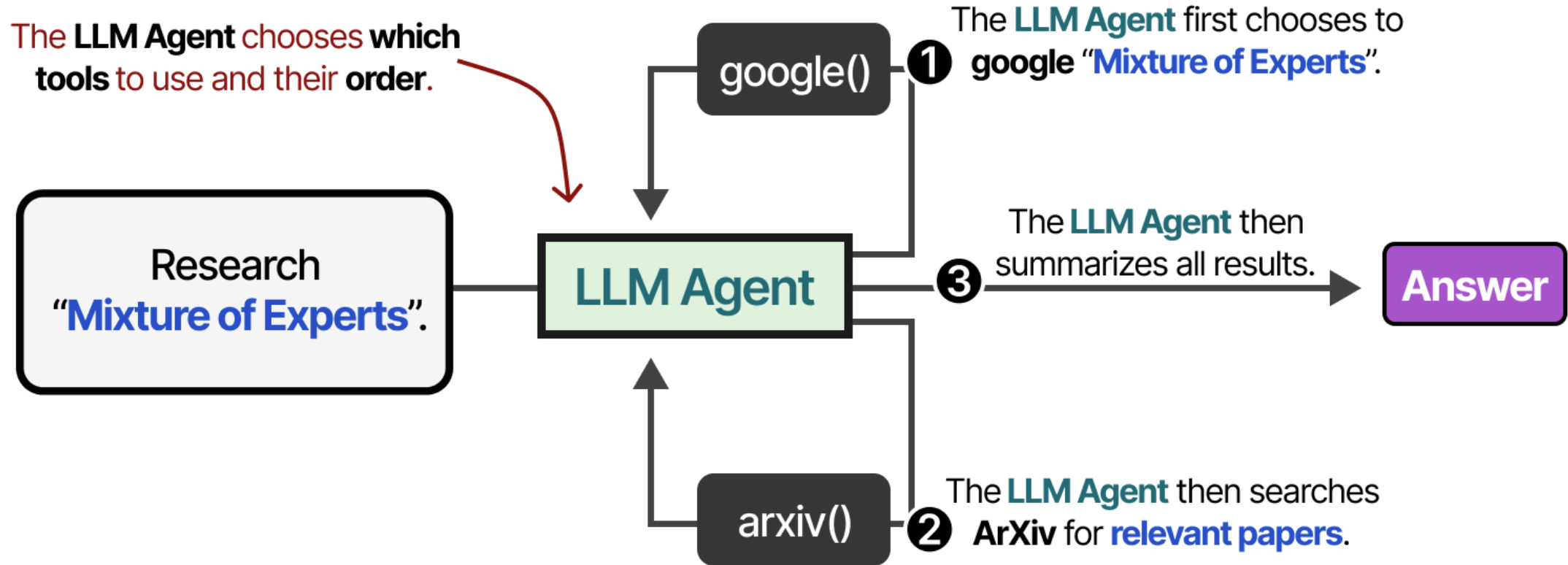
FIXED PIPELINE — tools in predetermined order

Ideal for routine, high-frequency tasks

- Auditable.
- Predictable.
- Every run is the same.

Fixed pipeline vs **Autonomous Choice** for Tool Calling

The LLM can follow a script or choose its own path for tool selection — most deployments sit in between

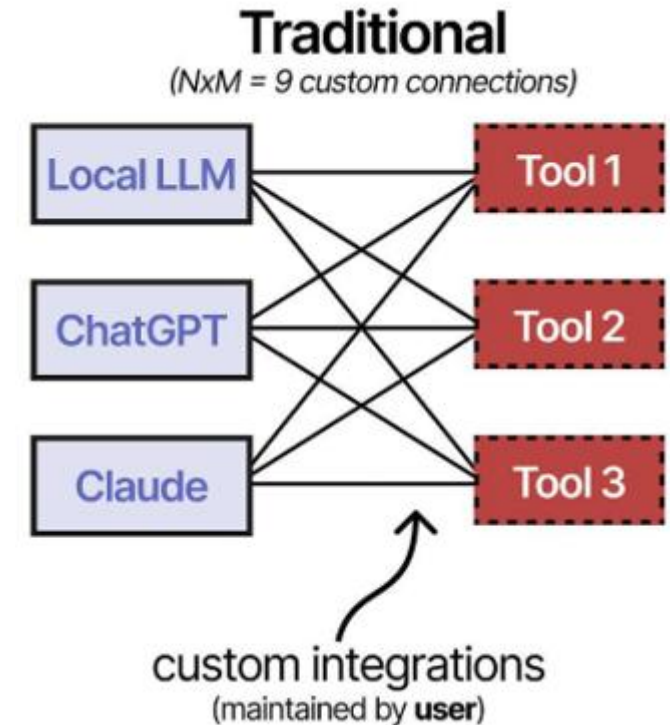
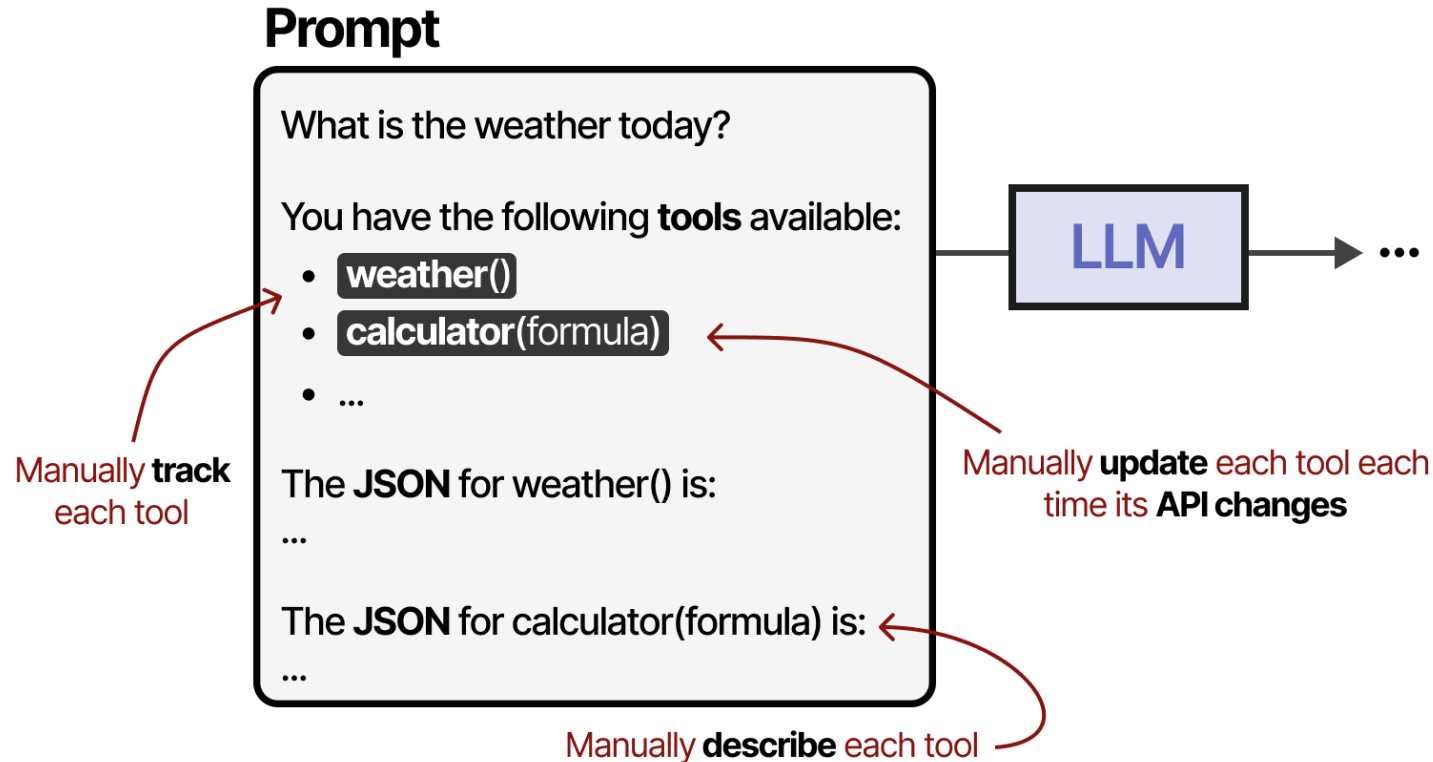


AUTONOMOUS SELECTION — LLM decides every step

- Powerful.
- Flexible.
- Requires thorough validation before deployment

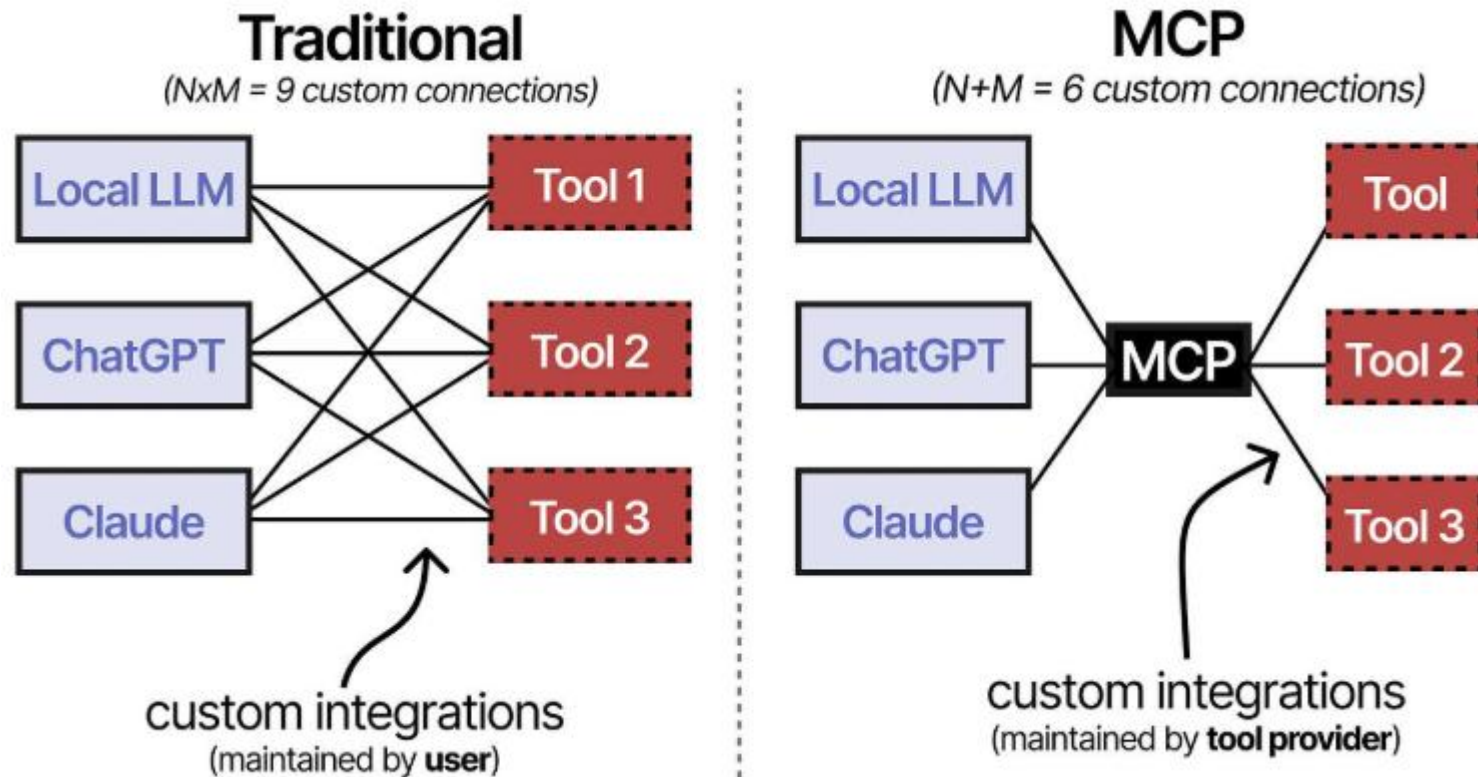
Need for a **standard C-Port** for tools to interact with an LLM

- Each tool needed a *custom integration* for each LLM application. As tools multiply, this becomes unmanageable.



Model Context Protocol (MCP)

- Each tool needed a *custom integration* for each LLM application. As tools multiply, this becomes unmanageable.
- MCP (created by Anthropic) creates a standard plug-any-tool, **one interface**



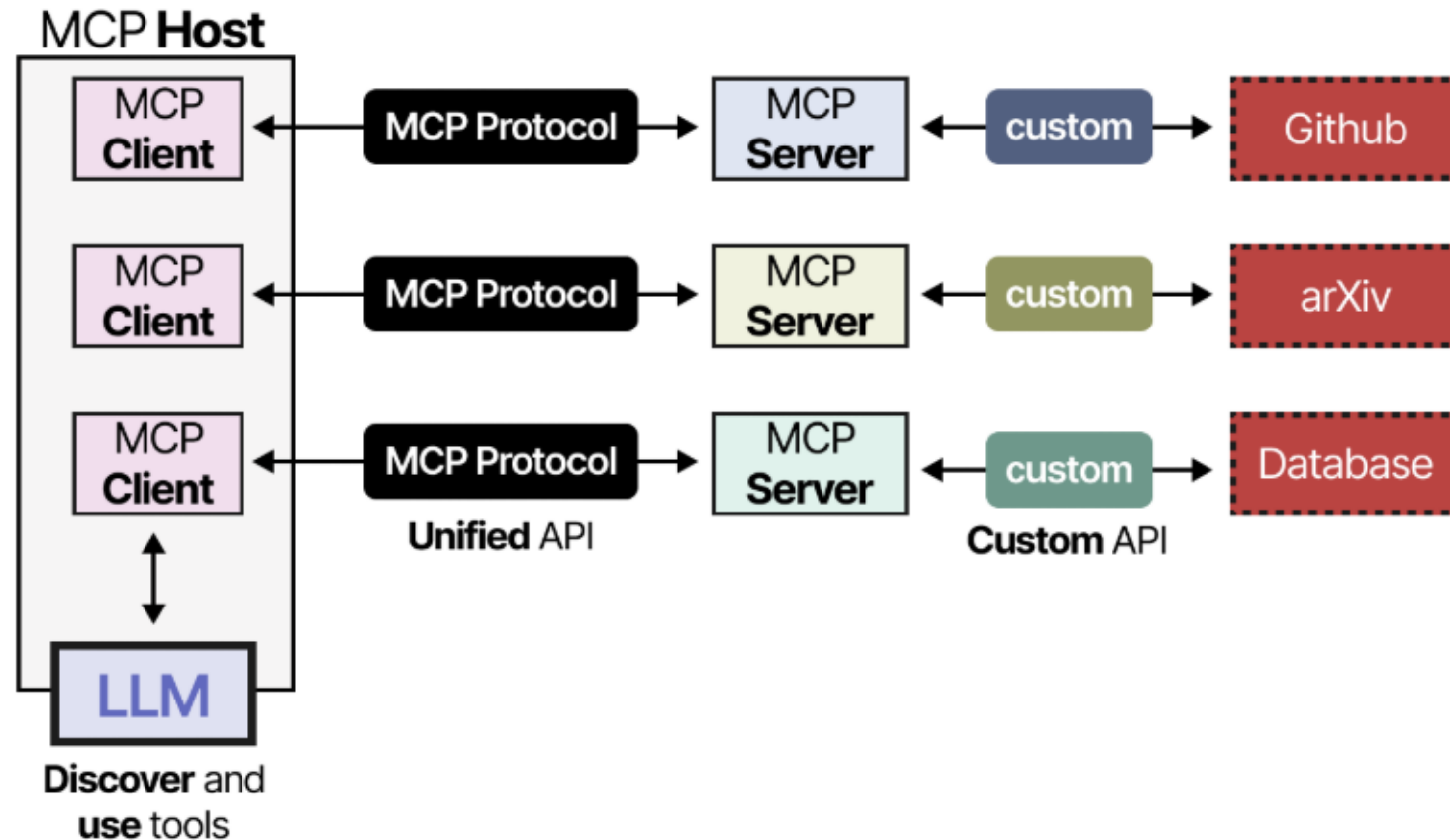
Model Context Protocol (MCP)

Three components of MCP

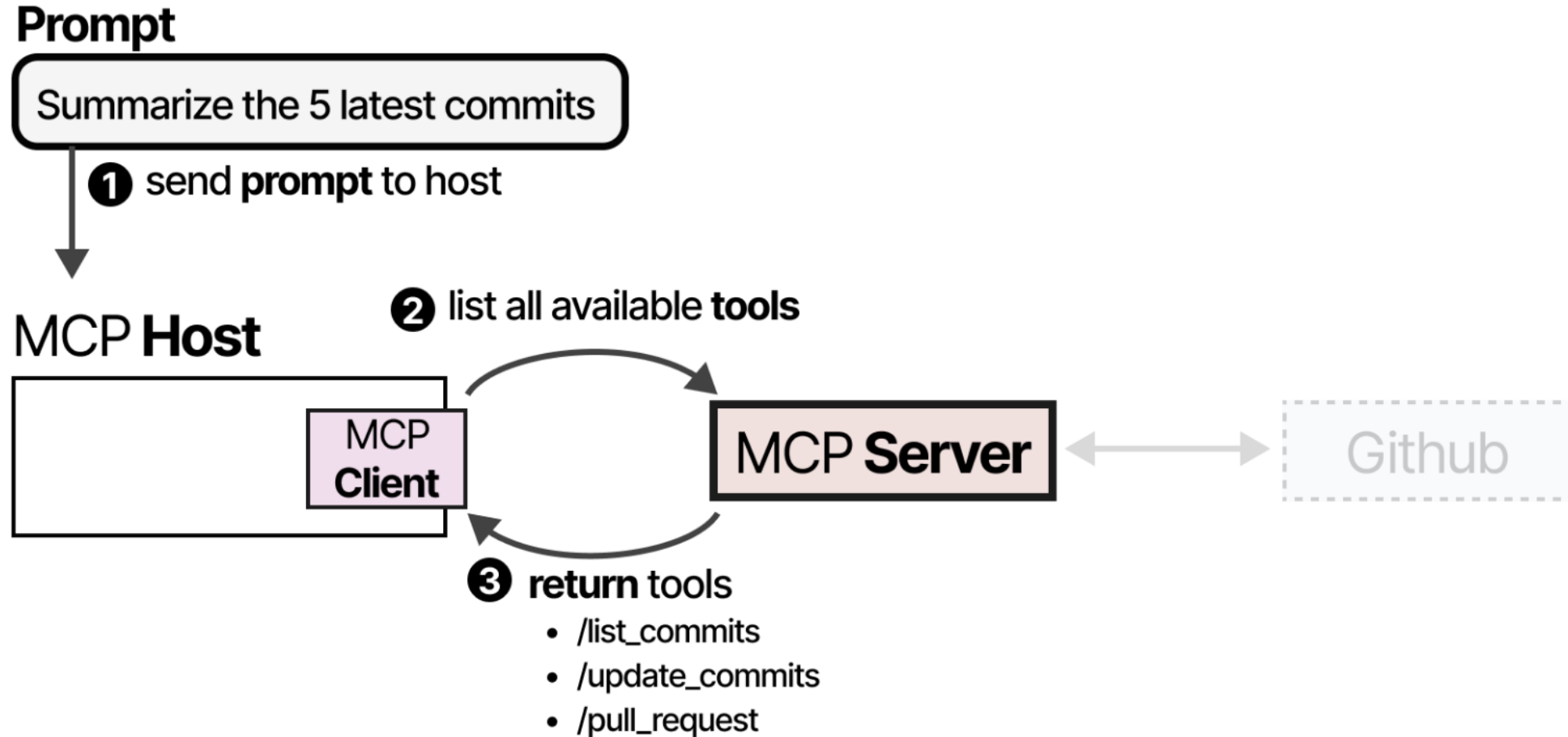
MCP Host: Runs the LLM (e.g. ChatGPT, Claude)

MCP Client: Stays within MCP hosts and maintains 1:1 connections with MCP servers

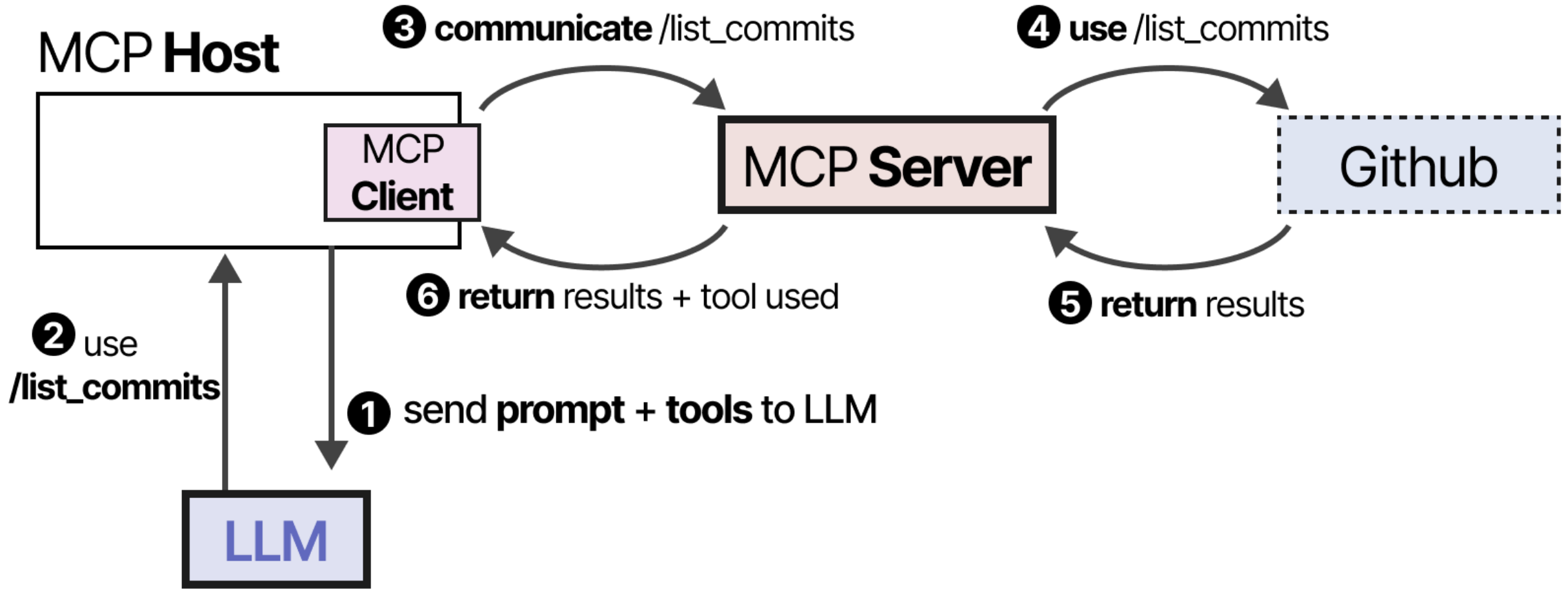
MCP Server: Exposes tools and data



Model Context Protocol (MCP) via example



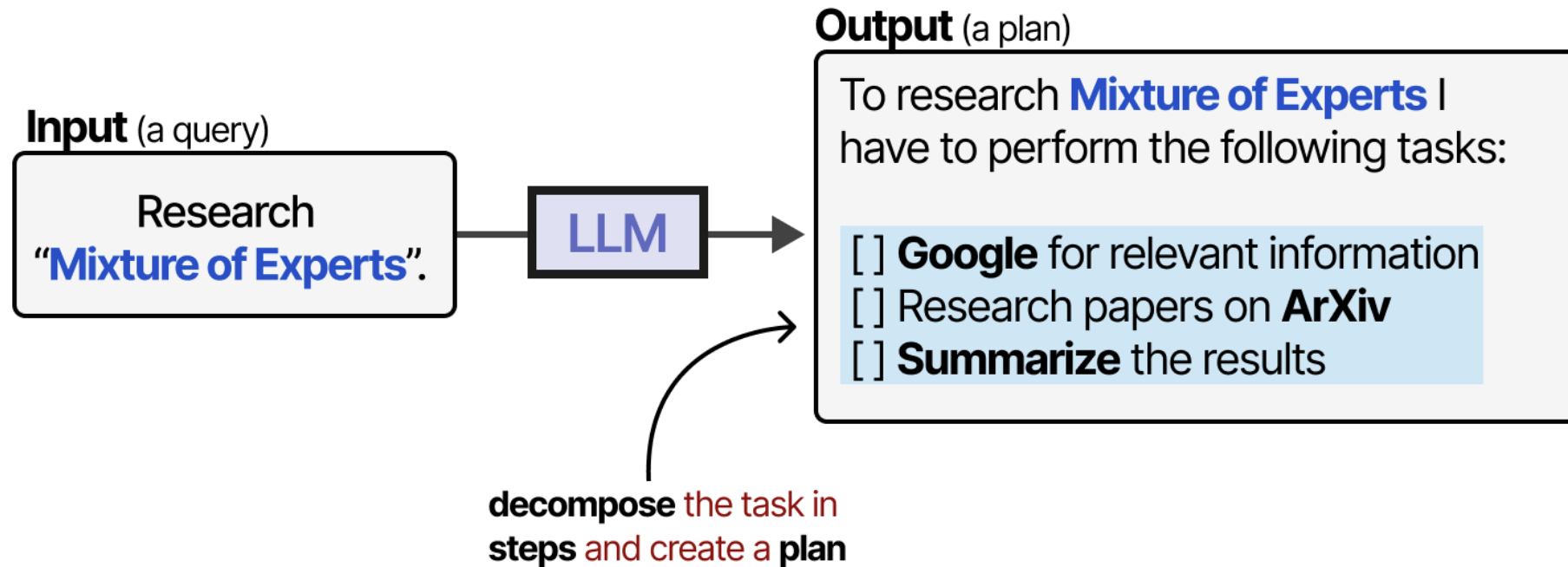
Model Context Protocol (MCP) via example



Planning

Planning: Breaking Goals into Sequence of Actions

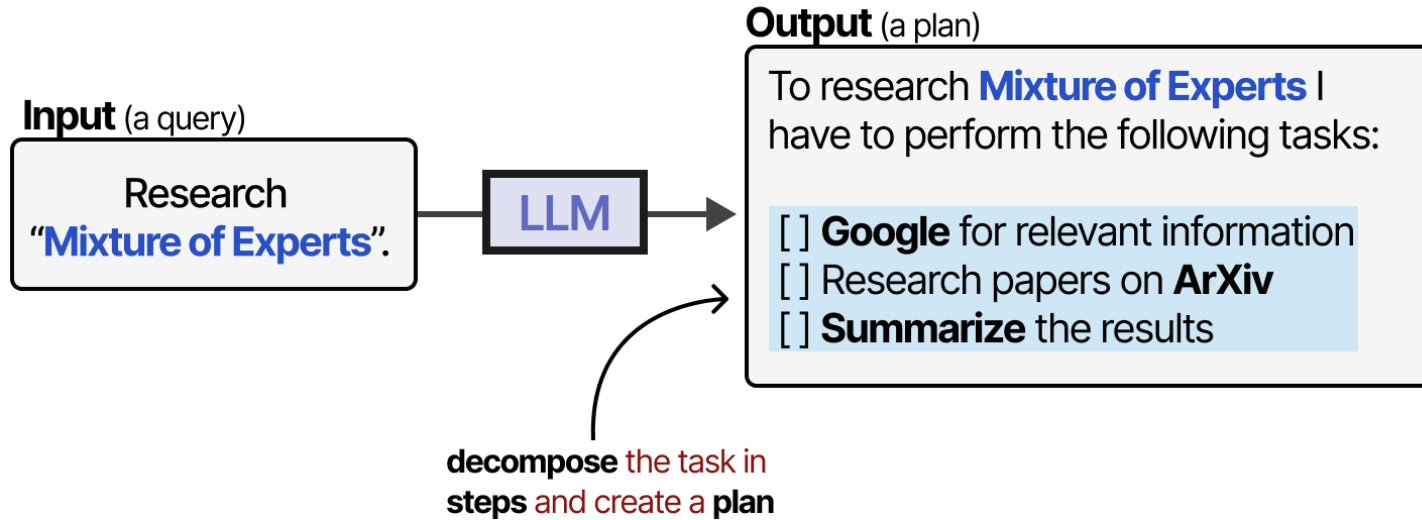
- Tool use allows an LLM to increase its capabilities. They are typically called using **JSON-like** requests.
- But how does the LLM, in an agentic system, decide which tool to use and when
- Planning in LLM Agents involves breaking a given task up into actionable steps



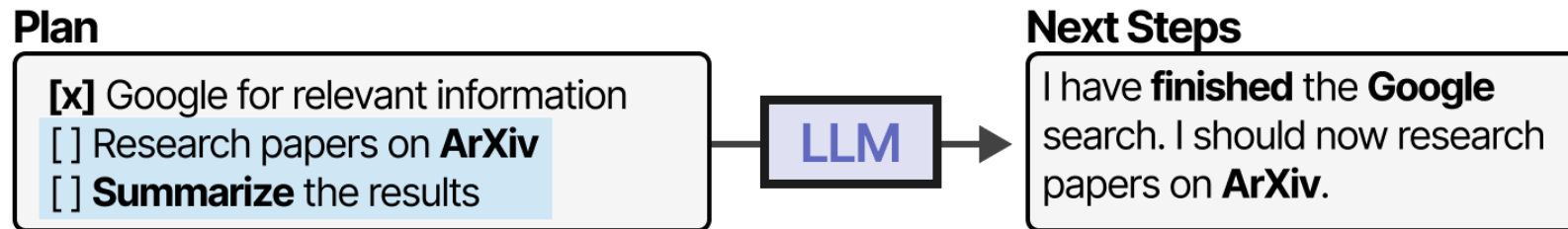
A high-level task decomposed into ordered steps, each with a clear action

Planning: Breaking Goals into Sequence of Actions

- Planning in LLM Agents involves breaking a given task up into actionable steps

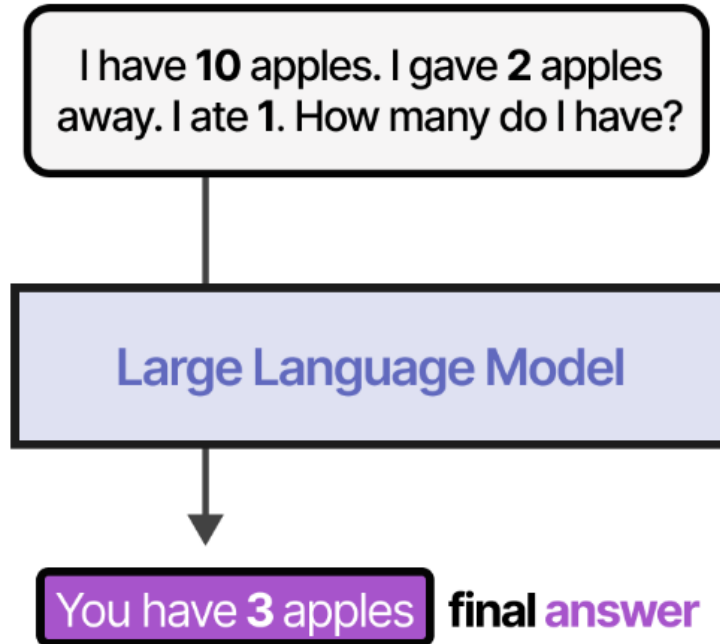


- This plan allows the model to iteratively reflect on past behavior and update the current plan if necessary.

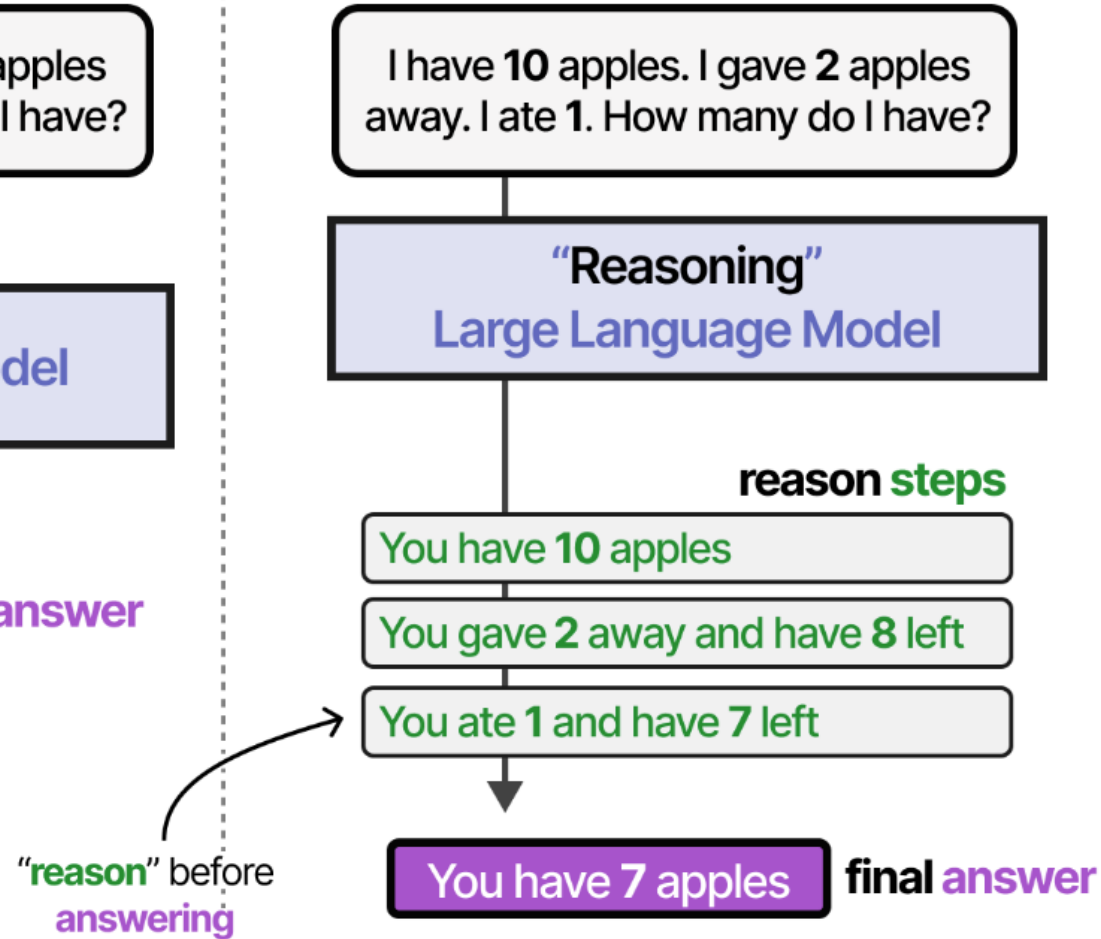


Reasoning for Planning: CoT

“Regular” LLMs



“Reasoning” LLMs



Reasoning for Planning: CoT

- Show the LLM how to think step by step, by prompting well

Reasoning prompt

Q: What is $3 + 2$?

Thoughts:

First, **3** and **1** gives **4**.

Then, **4** and **1** gives **5**.

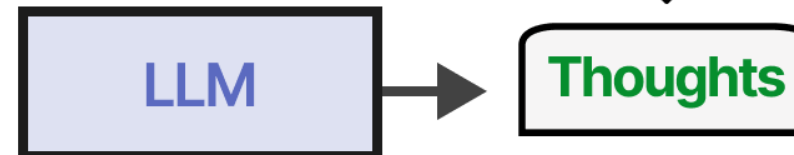
I believe the answer is **5**.

Answer: **5**

Q: What is $9 + 1$?

New question

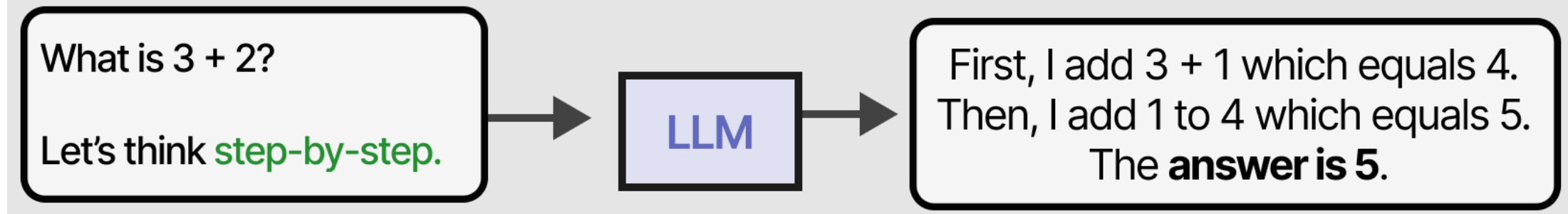
This methodology of providing examples of Chain-of-Thought and enables more complex reasoning behavior.



Reasoning for Planning: CoT

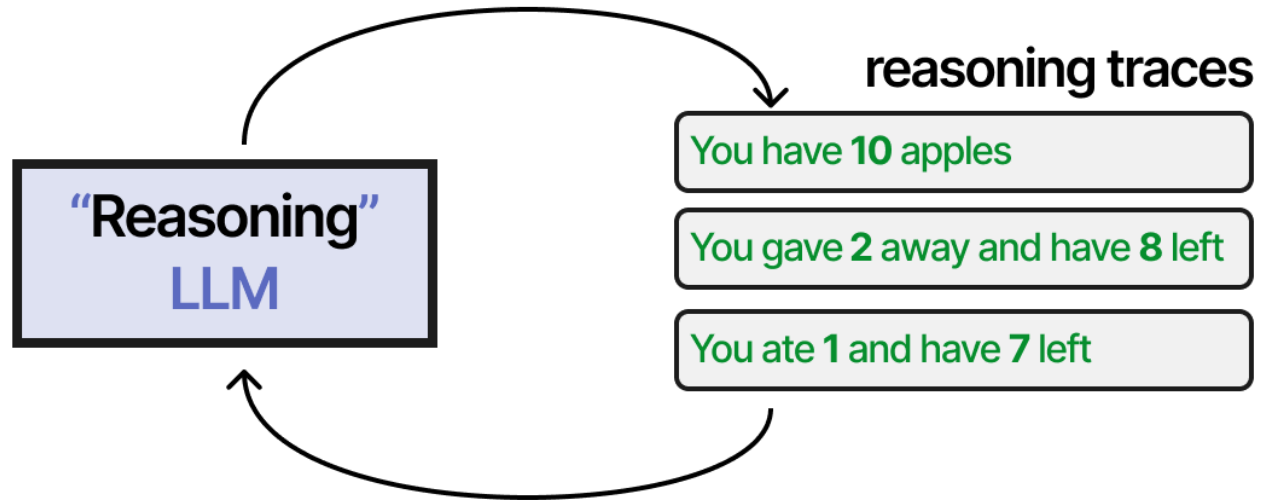
- Chain-of-thought can also be enabled without any examples (zero-shot prompting) by simply stating “Let’s think step-by-step”

implicitly enabling reasoning



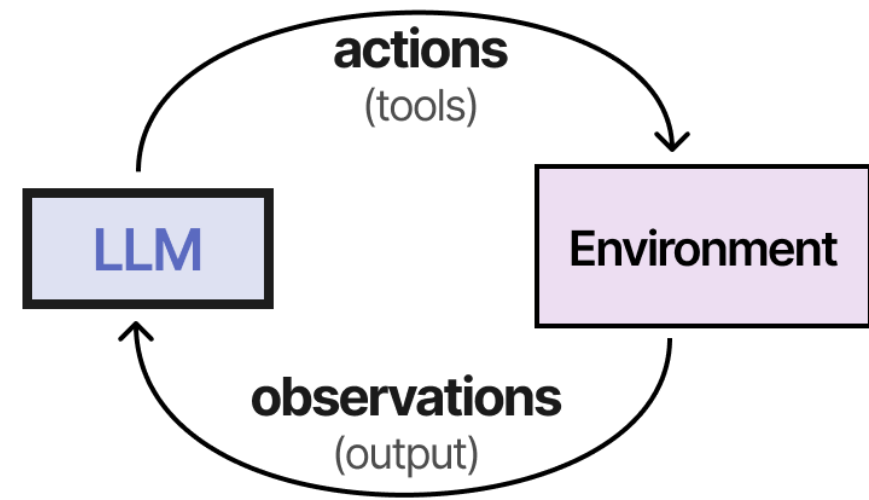
- When training an LLM, we can either give it a sufficient amount of datasets that include thought-like examples or the LLM can discover its own thinking process.

ReAct: Reason and Act



Reasoning Only
(e.g., chain-of-thought)

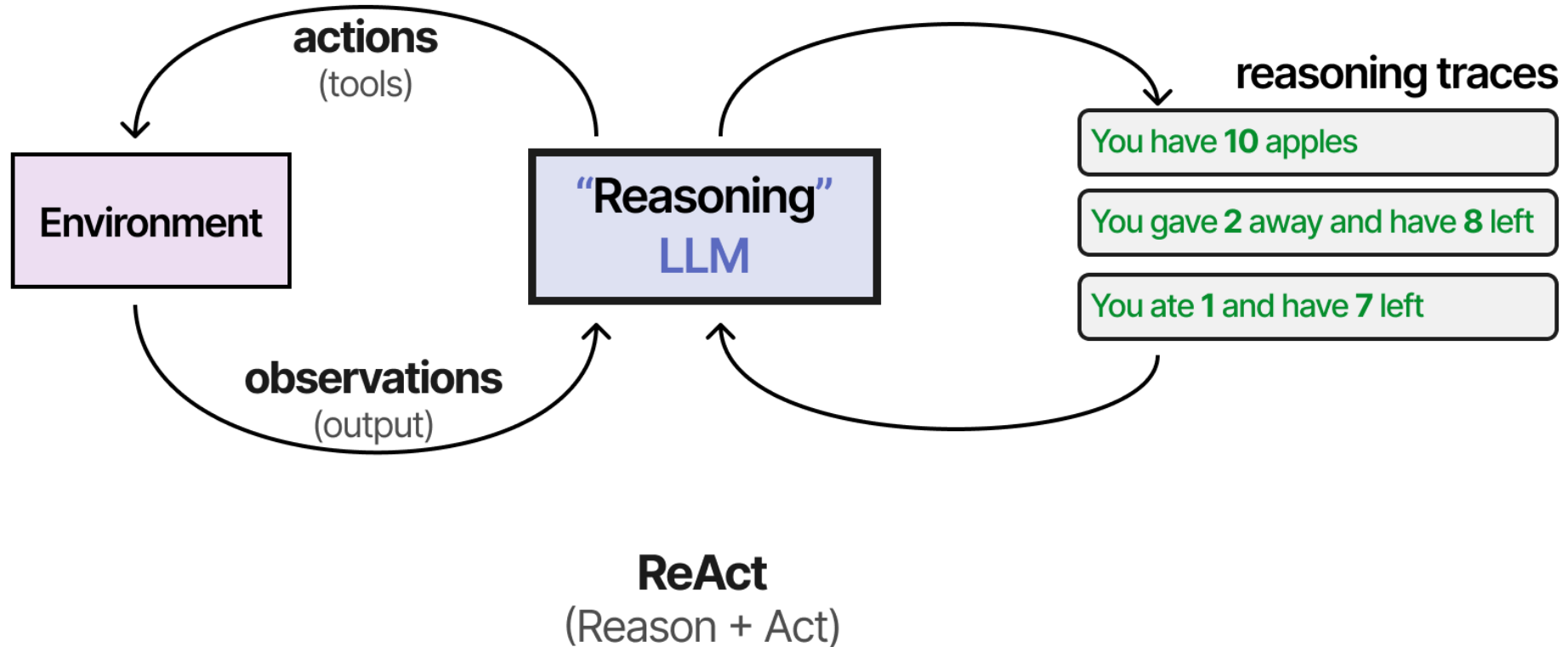
No action



Act Only
(e.g., Toolformer)

No reasoning

ReAct: Reason and Act



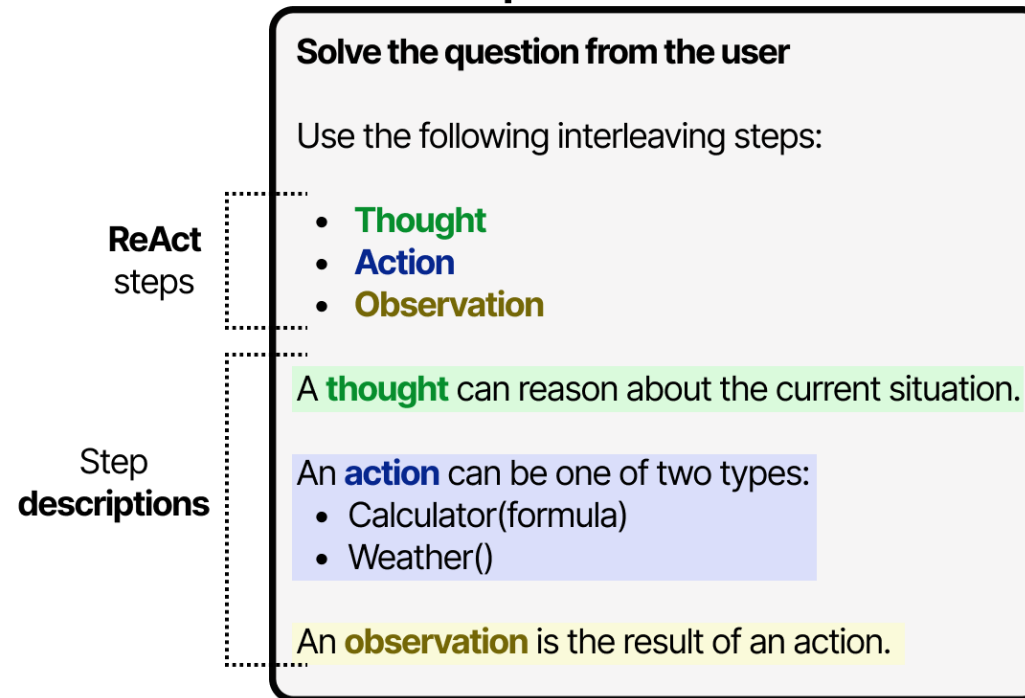
ReAct fills the gap: reason, then act on what you reasoned, then reason again based on what you observed

ReAct in Detail

- **Thought:** The LLM writes its reasoning. What does it know? What does it need?
- **Action:** The LLM specifies a tool call. What tool and with what arguments?
- **Observation:** The tool result is returned. What does this tell us? Does the plan change?

System Prompt (not user)

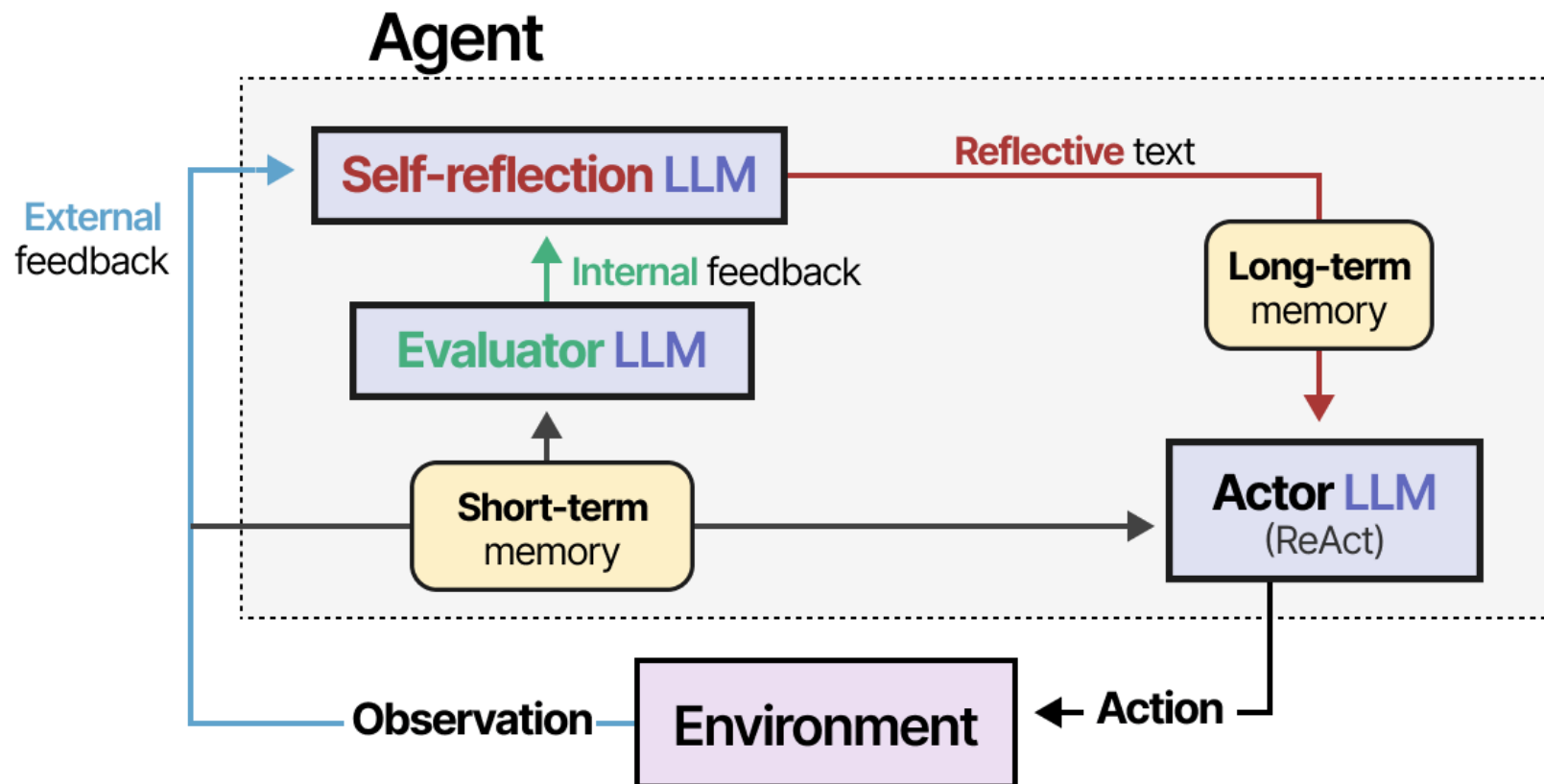
ReAct Prompt



Reflexion: Learning from Failure

Reflexion adds a self-evaluation loop — the agent learns from its own mistakes

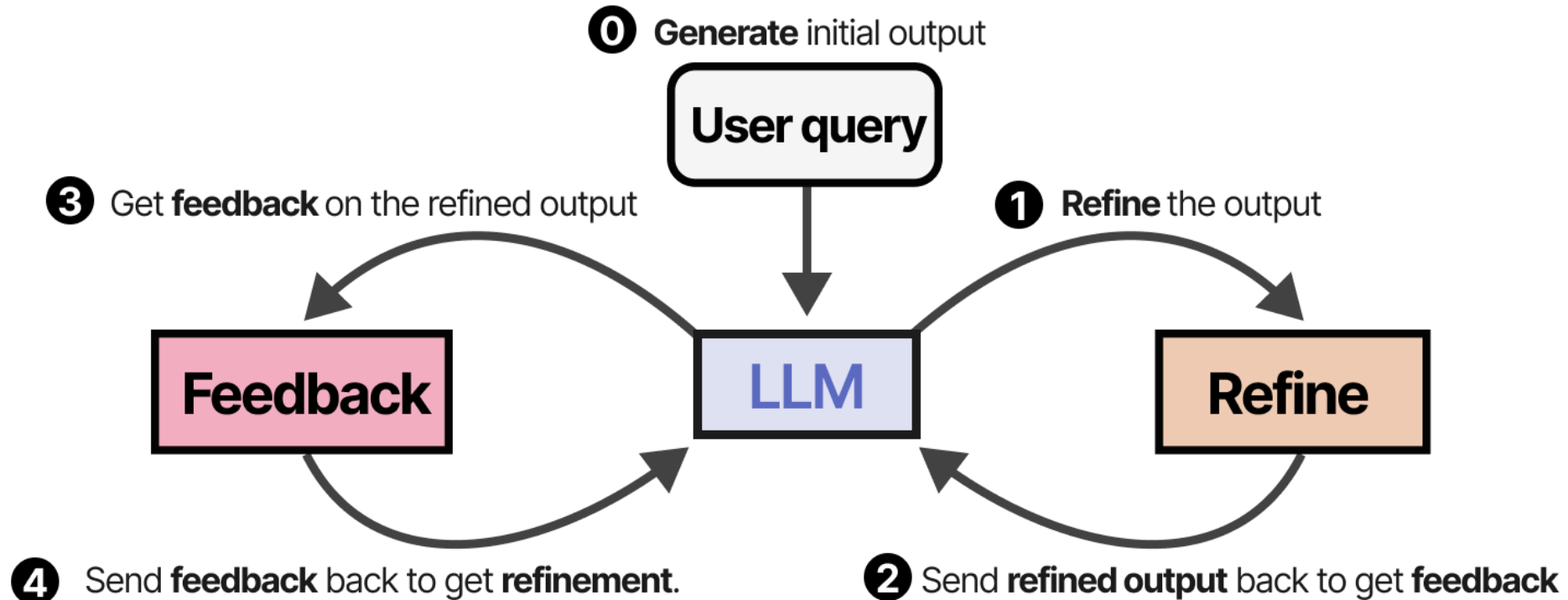
- **Actor LLM:** Executes actions. Uses ReAct or chain-of-thought to choose steps.
- **Evaluator LLM:** Scores the Actor's output. Did the actions achieve the goal? How well?
- **Self-reflection LLM:** Converts the score into a lesson. What should the Actor do differently next time?



Memory modules are added to track actions (short-term) and self-reflections (long-term), helping the Agent learn from its mistakes and identify improved actions.

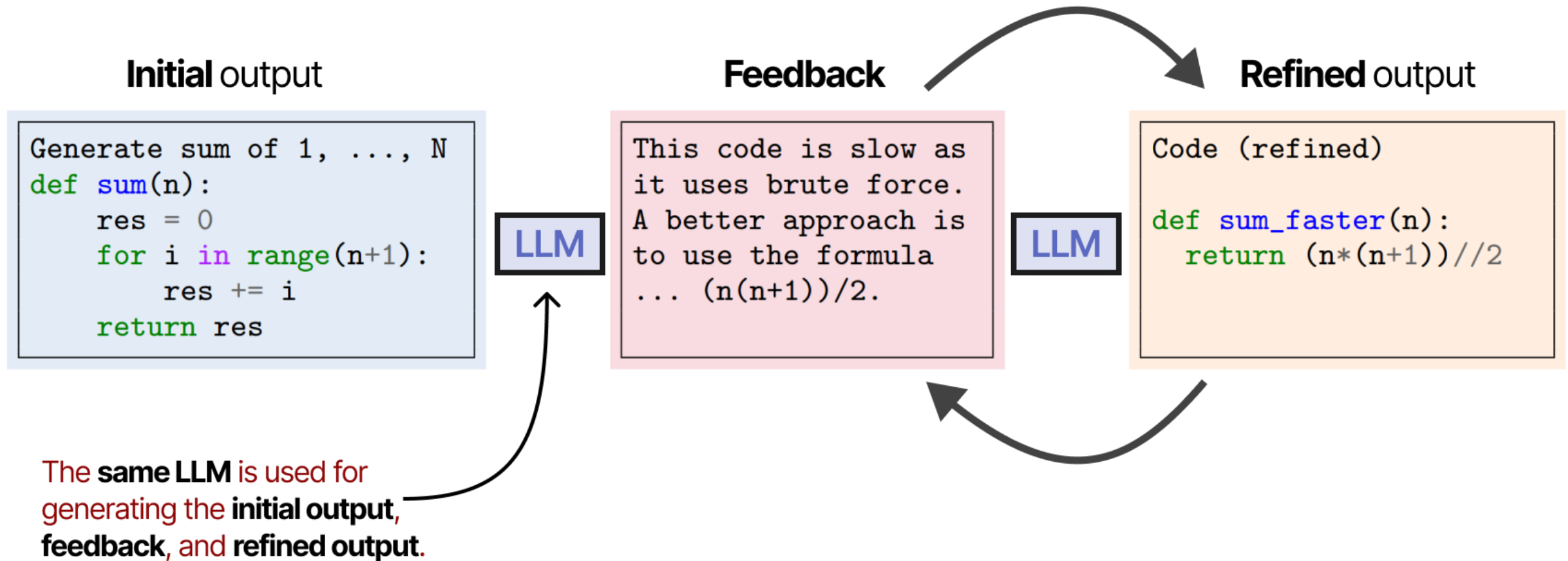
Self-Refine: Iterative Improvement

SELF-REFINE: **one** LLM generates an answer, then critiques it, and then rewrites it



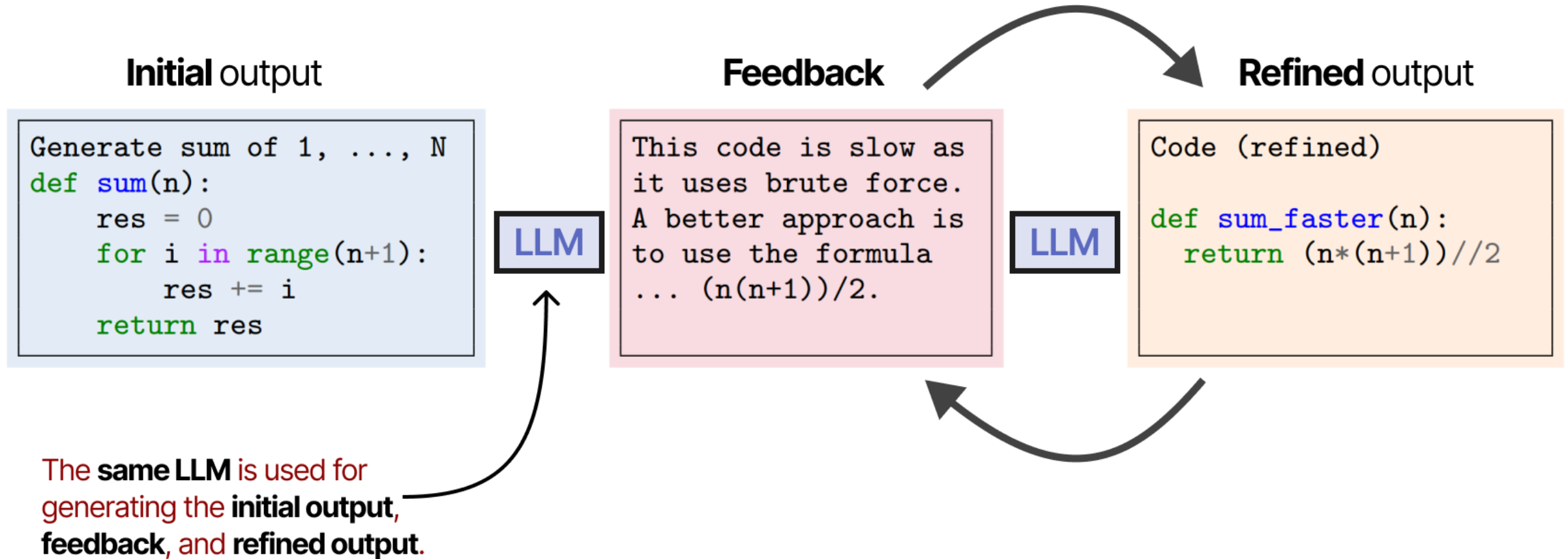
Self-Refine: Iterative Improvement

Example



Self-Refine: Iterative Improvement

Example

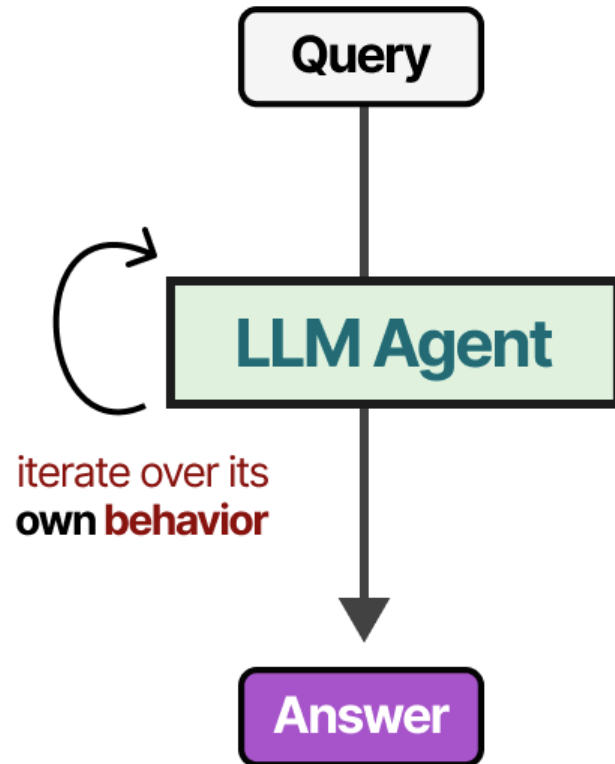


Multi-Agent Systems

Why Multi-Agent?

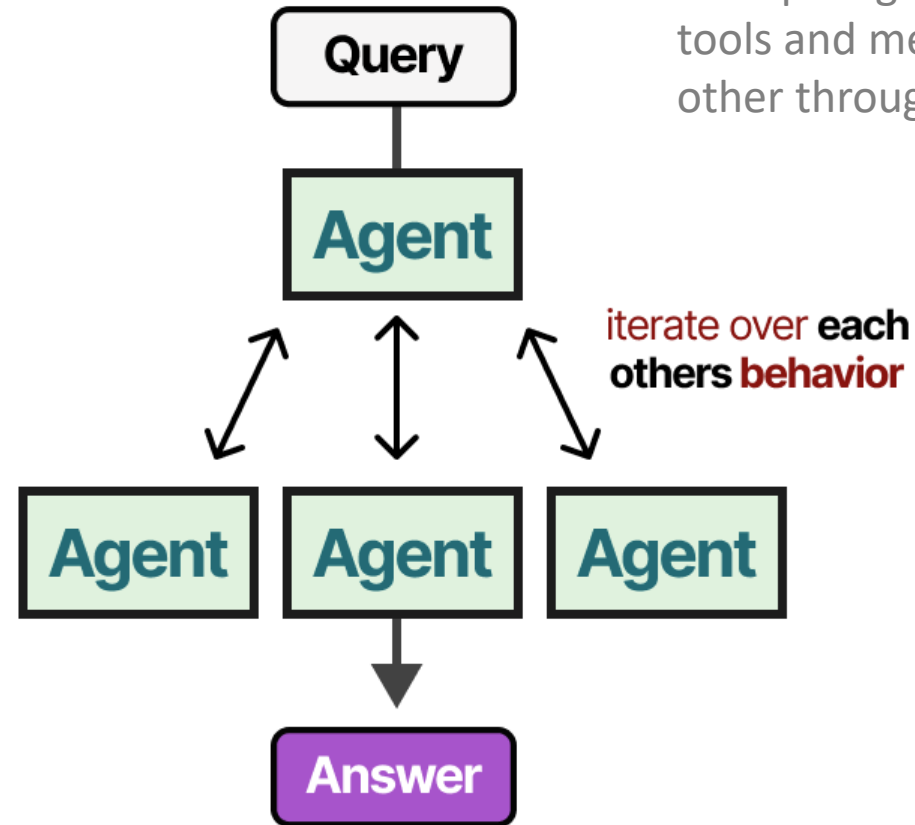
Too many tools in a single agent leads to poor tool selection decisions.

Single Agent



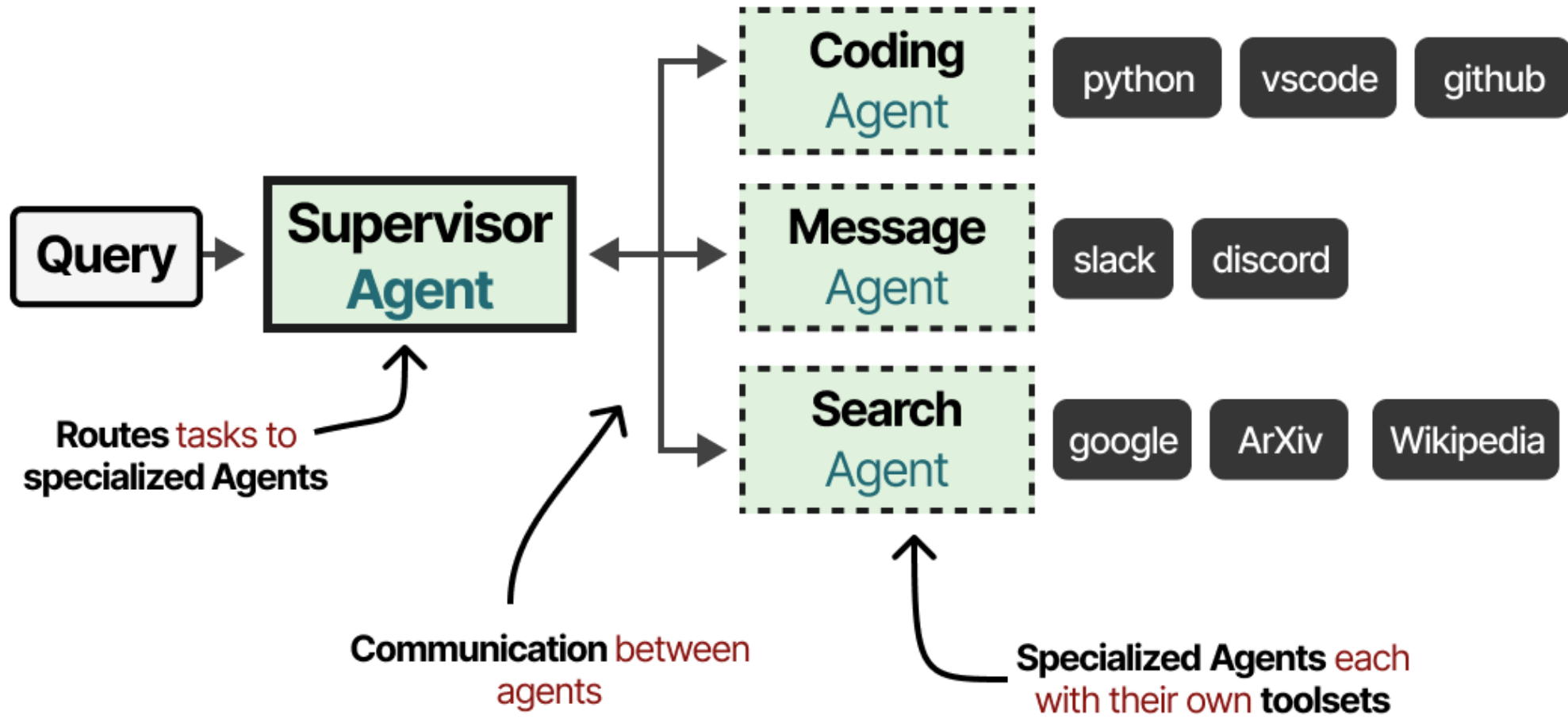
Multi-Agent

Multiple agents each with their own tools and memory, connected to each other through shared communication



Multi-Agent: Supervisor-Specialist Structure

A supervisor coordinates specialists — each expert in their own domain



Supervisor agent at top, specialist agents below with their own dedicated tool sets

Multi-Agent: Supervisor-Specialist Structure

Initialization (how each agent is set up) and **Orchestration** (how they coordinate) are two separate design problems

